BVS1 – Betriebssysteme

Winter Semester 2026/2027 · TH Köln · ICCT

Stavros Kousidis

Table of contents

Course Overview **6**
Course Material . . . 6
Use of Large Language Models . . . 6
License . . . 6

1 From Executing Code to Operating Systems **7**
1.1 One program can run without a guest operating system . . . 7
1.1.1 Interpreting the experiment . . . 8
1.2 The ISA connects software and processor . . . 9
1.2.1 Why RV64? . . . 9
1.3 Development host and target are different roles . . . 9
1.4 Virtualization and emulation are different . . . 10
1.4.1 Virtualization: the guest instructions already belong to the host ISA . . . 10
1.4.2 Emulation: the target instruction may not be a host instruction . . . 11
1.4.3 Our BVS1 machine stack . . . 11
1.5 The toolchain connects development host and target . . . 12
1.5.1 Object files are not yet the complete program . . . 13
1.6 When does an operating system become useful? . . . 13
1.7 Applying the working definition . . . 14
1.8 From Hello BVS1 to a minimal operating system . . . 15
1.9 Key ideas . . . 16
Related Exercise . . . 17
References . . . 17

2 From Source to Executables: Towards Processes **18**
2.1 From source code to target-specific object code . . . 18
2.2 Why object files still need a linker . . . 19
2.3 From the stored-program idea to a concrete memory layout . . . 20
2.4 Linking produces the program QEMU can load . . . 21
2.5 Why execution starts before `main` . . . 21
2.6 The whole path, and what Make hides . . . 22
2.7 Change, rebuild, observe . . . 22
2.8 Why an executing program is not yet a process . . . 23
2.9 Yield, exit, resume, and restart . . . 24
2.10 The question that leads to execution context . . . 24
2.11 Key ideas . . . 25
Related Exercise . . . 26
References . . . 26

3 First Process: Stop and Continue **27**
3.1 C pointer notation: a reminder . . . 27
3.2 Building the process object . . . 28
3.2.1 Entry function . . . 28
3.2.2 Owned stack storage . . . 28
3.2.3 Lifecycle information . . . 29
3.2.4 What is still missing? . . . 29
3.3 Understanding the process stack . . . 30
3.4 Continuation: where should execution return? . . . 31
3.5 Context . . . 32
3.6 The complete process object . . . 32
3.7 The provided context-switch boundary . . . 33
3.8 Implementation boundary . . . 34
3.9 Public process interface . . . 35
3.10 The lifecycle in C . . . 35
3.10.1 Create / prepare . . . 35
3.10.2 Run and resume . . . 36
3.10.3 Yield . . . 37
3.10.4 Exit . . . 37
3.11 The complete CP1 observation . . . 37
3.12 Why CP1 deliberately uses yield . . . 38
3.13 Check your model . . . 38
3.14 Key ideas . . . 39
Related Exercise and Checkpoint . . . 39
References . . . 39

4 Where Our Code Runs **40**
4.1 Locate editing, building, and execution . . . 40
4.2 Establish a reliable edit–build–run loop . . . 41
4.3 One managed execution can contain several active calls . . . 42
4.4 Instructions and runtime data have different roles . . . 42
4.5 `sp`, `ra`, and `pc` are different kinds of CPU state . . . 43
4.6 Follow the same execution through yield and resume . . . 44
4.7 Connect the other memory regions . . . 44
4.8 The process stack is an array in an object . . . 45
4.9 Inspect the file without confusing it with live memory . . . 46
4.10 The next requirement: regain control without cooperation . . . 46
4.11 Key ideas . . . 47
Related Exercise and Handouts . . . 47
References . . . 48

5 Entering the OS **49**
5.1 The mental model of our current OS . . . 49
5.1.1 The lifetime from Lecture 3, revisited . . . 49
5.1.2 What if the process never yields? . . . 50

5.2 From calls to traps . . . 50
5.2.1 Two halves of the answer . . . 50
5.2.2 The lifetime through the CPU . . . 50
5.2.3 What the CPU provides for traps and returns . . . 51
5.3 Trap and return: what the CPU does . . . 52
5.3.1 Why a function call is not enough . . . 52
5.3.2 Traps . . . 52
5.3.3 Who can save the `pc`? . . . 53
5.3.4 On a trap: record where and why . . . 54
5.3.5 On a trap: interrupts off . . . 55
5.3.6 On a trap: jump . . . 55
5.3.7 On a return: `mret` . . . 56
5.4 Between trap and return: the OS's mechanisms . . . 56
5.4.1 Why `ra`, `sp`, `s0`–`s11` no longer suffice . . . 56
5.4.2 The trap frame replaces `struct context` . . . 58
5.4.3 The trap path replaces `context_switch()`: save . . . 59
5.4.4 The return path replaces `context_switch()`: restore . . . 61
5.4.5 `trap_handler()`: return the frame to restore . . . 62
5.5 Refactoring `main()` and `process_yield()` to use traps . . . 63
5.5.1 `ecall`: a trap on purpose . . . 63
5.5.2 Continue after the `ecall` . . . 63
5.5.3 `yield` and `exit` as system calls . . . 64
5.5.4 Inside `trap_handler()` . . . 64
5.5.5 Unexpected traps . . . 65
5.5.6 First entry: prepare a frame . . . 65
5.5.7 Three entries . . . 66
5.5.8 Why `p` must become static . . . 67
5.6 The whole path . . . 67
5.6.1 The lifetime in detail . . . 67
5.6.2 Our trap model: who does what . . . 68
5.7 Key ideas . . . 69
Related Exercise . . . 69
References . . . 70

6 Timer and Preemption 71
6.1 Where we stand . . . 71
6.1.1 The lifetime through the CPU, recalled . . . 71
6.1.2 Why a never-yielding loop is still stuck . . . 72
6.1.3 Two halves of the answer . . . 72
6.2 Time as an event: the platform timer . . . 72
6.2.1 A device you have used since Exercise 1: the UART . . . 72
6.2.2 The timer is a platform device . . . 73
6.2.3 `mtime` and `mtimecmp` . . . 74
6.2.4 Reading and writing a device register in C . . . 74
6.2.5 The time slice . . . 76
6.2.6 Pending: a request, not yet a trap . . . 76

6.3 Letting the interrupt in . . . 77
6.3.1 Three conditions for taking a timer interrupt . . . 77
6.3.2 Two switches in series: `mie.MTIE` and `mstatus.MIE` . . . 77
6.3.3 Who sets which bit in our system . . . 78
6.3.4 The global switch over time: the payoff of "interrupts off" . . . 79
6.3.5 What held in Exercise 4 . . . 79
6.4 The same trap path, now asynchronous . . . 80
6.4.1 The CPU takes the interrupt . . . 80
6.4.2 `mcause`: interrupt bit first . . . 80
6.4.3 The trap `pc` is the continuation address . . . 81
6.4.4 The timer branch of `trap_handler()` . . . 82
6.4.5 The lifetime with a timer interrupt . . . 83
6.5 Preemption . . . 84
6.5.1 Losing the CPU without yielding . . . 84
6.5.2 Interrupt handling versus the scheduling decision . . . 84
6.5.3 One line per tick . . . 85
6.5.4 Can the process tell it was interrupted? . . . 85
6.5.5 Reproducible interrupts with `-icount` . . . 86
6.6 What can go wrong . . . 86
6.6.1 Ticks without progress . . . 86
6.6.2 Our trap model: who does what, with the timer . . . 87
6.7 Key ideas . . . 87
Related Exercise . . . 88
References . . . 88

Appendices **89**

A Glossary **89**

Course Overview

This site contains the teaching material for **BVS1 – Betriebssysteme** at TH Köln, ICCT, for **Winter Semester 2026/2027 · TH Köln · ICCT**. The course is attended by students from multiple Bachelor programmes.

The course provides two aligned representations: concise lecture-specific Beamer slides for classroom interaction, and a reader-oriented chapter source used for both the PDF script and the HTML course.

Course Material

The lecture chapters are listed in the HTML navigation and in the PDF table of contents.

Use of Large Language Models

These teaching materials were developed with the assistance of large language models for drafting, restructuring, review, consistency checking, and technical and pedagogical refinement. The instructor remains responsible for the selection, verification, organization, and content of the material.

License

Except where otherwise noted, the student-facing BVS1 teaching materials are © 2026 Stavros Kousidis and licensed under Creative Commons Attribution-ShareAlike 4.0 International (CC BY-SA 4.0).

Third-party material remains subject to its respective license or copyright terms. The separately distributed BVS1 software repository `teaching-bvs1-code` remains licensed under the MIT License.

1 From Executing Code to Operating Systems

An operating system is familiar enough that it is easy to begin with examples rather than with the problem it solves. Windows, Ubuntu, Android and macOS are recognizable operating-system environments, but naming familiar systems does not yet explain why operating systems exist.

BVS1 therefore starts with a smaller question:

Can a computer execute a program without an operating system?

The first practical system is deliberately tiny. We use QEMU to provide a machine with an **RV64 (64-bit RISC-V)** CPU. This is our **target**: the machine for which we build the program. It runs one program and prints:

```
Hello BVS1
```

The experiment gives us a concrete machine to reason about before we introduce operating-system mechanisms.

1.1 One program can run without a guest operating system

At the lowest level, a processor executes instructions. Those instructions operate on registers and memory and can communicate with devices.

One important idea of the stored-program / Von Neumann model is that **instructions and data are both kept in memory**.

A deliberately simplified machine model is:

```
    instructions + data
           |
           ▼
        memory
           ▲
           |
           ▼
          CPU
           |
           ▼
```

```
        devices
```

This model omits many details of real hardware, but it is enough for the first distinction:

> **CPU, memory and devices exist independently of an operating system.**

The point here is conceptual rather than historical or microarchitectural. Modern processors may use separate instruction and data caches internally, but the important point here is that instructions and data both occupy target memory. Lecture 2 returns to this idea and makes the abstract memory model concrete.

In the `Hello BVS1` experiment, QEMU provides the target machine. The target contains an RV64 CPU, memory and devices. The program is loaded and begins executing without a guest operating system.

The physical computer still runs an operating system, and our Ubuntu development environment also runs an operating system. The important observation concerns the **target**:

> **Executing instructions is not, by itself, what makes an operating system necessary.**

1.1.1 Interpreting the experiment

The classroom experiment deliberately separates observation from explanation. Consider these statements:

- QEMU executed our program.
- A virtual CPU executed our program.
- Ubuntu executed our guest program.
- The target had an operating system.
- QEMU was already running before our guest program started.
- The string `Hello BVS1` existed somewhere in memory while the program was running.

The purpose of discussing them is to expose the execution model.

A precise description is:

- the Ubuntu development VM runs the QEMU process;
- QEMU implements the target machine;
- the target contains an emulated RV64 CPU, memory and devices;
- the RV64 program runs on that target;
- the target has no guest operating system.

Saying "QEMU executed the program" is understandable in informal language, but the more useful model is that QEMU **emulates the machine whose CPU executes the target instructions**.

1.2 The ISA connects software and processor

Machine code only has meaning if software and the processor agree on what each instruction means.

This agreement is the **Instruction Set Architecture (ISA)**.

An ISA specifies properties visible to machine-level software, including:

- instructions and their encodings,
- registers,
- memory-addressing rules,
- exceptional control transfers,
- privilege mechanisms.

Different processor implementations can implement the same ISA.

The ISA should therefore be distinguished from the **microarchitecture**:

- **ISA:** the contract visible to software;
- **microarchitecture:** how a particular processor implements that contract internally.

For BVS1, the teaching target uses **RV64**, the 64-bit form of RISC-V.

1.2.1 Why RV64?

RISC-V is a teaching choice, not a claim that it is the best ISA for every system.

It offers a comparatively regular architecture, public specifications, a modern privilege model, suitable development tools, and a well-supported QEMU `virt` machine. These properties let us expose operating-system mechanisms without making historical processor complexity the subject of the course.

1.3 Development host and target are different roles

The machine on which software is developed does not need to use the same ISA as the machine for which the program is built.

In BVS1:

- the **development host** is the Ubuntu `bvs1` VM in which the development tools run;
- the **target** is the RV64 machine for which we build the program.

The development VM may itself run on an x86-64 or Arm64 host platform. The teaching target is nevertheless RV64.

This gives two separate questions:

1. Where do the development tools execute?
2. Which machine will execute the generated program?

Keeping these roles separate will matter when we build code on one machine for another and when we examine emulation.

1.4 Virtualization and emulation are different

The most useful first question is:

Can the physical CPU execute the guest or target instructions directly?

That question separates the roles of virtualization and emulation more clearly than the word *virtual machine* by itself.

1.4.1 Virtualization: the guest instructions already belong to the host ISA

In hardware-assisted virtualization, guest and host normally use the same processor ISA.

A guest instruction is therefore, in the ordinary case, already an instruction that the physical CPU understands. Most non-privileged guest instructions can execute directly on the physical processor.

The virtualization mechanism controls the parts that must not simply be handed to the guest, for example:

- isolation between machines,
- privileged processor state,
- access to memory and devices,
- transitions that require intervention by the virtualization layer.

A simplified model is:

```
guest instruction
       ↓
physical CPU
```

The important point is not that virtualization performs no mediation. It is that the guest machine does not require a different instruction vocabulary from the physical CPU.

In BVS1, Multipass manages the Ubuntu development VM using virtualization facilities available on the host platform. The exact backend can differ between operating systems.

1.4.2 Emulation: the target instruction may not be a host instruction

Emulation can reproduce a machine whose ISA differs from the host ISA.

Suppose the physical processor uses x86-64 while the BVS1 target uses RV64. The physical x86-64 CPU does not directly understand an RV64 instruction.

An emulator must therefore reproduce the effect of the target instruction. Conceptually:

```
RV64 target instruction
          ↓
        QEMU
          ↓
interpret / translate
          ↓
host operations
          ↓
physical CPU
```

The word **interpret** is useful for the conceptual distinction, but it should not be understood as a claim that an emulator must process every instruction individually. QEMU can dynamically translate blocks of target instructions into host code.

The essential point is:

> **Virtualization can largely run guest instructions directly; emulation must reproduce what target instructions would do when the host cannot execute them directly.**

1.4.3 Our BVS1 machine stack

The resulting stack is:

```
physical host
     ↓  virtualization (managed through Multipass)
Ubuntu development VM: bvs1
     ↓  runs
QEMU
     ↓  emulates RV64
RV64 target machine
     ↓  executes
Hello BVS1
```

The layers have different roles:

- the **physical host** provides the real CPU, memory and devices;

- the **Ubuntu development VM** is the common environment in which the course tools run;
- **Multipass** manages that development VM and is not an additional machine layer;
- **QEMU** runs as a normal program inside the development VM;
- QEMU emulates the **RV64 target machine**;
- `Hello BVS1` executes on that target.

Only the RV64 target is the teaching machine whose operating-system mechanisms we will build. It initially contains no guest operating system.

A **virtual machine** is the machine environment presented to software. Virtualization and emulation are different techniques that can provide such environments.

QEMU itself is capable of more than the configuration used here: in other settings it can also participate in hardware-assisted virtualization. In BVS1, its relevant role is RV64 emulation.

1.5 The toolchain connects development host and target

The source program is written in a human-readable programming language, but the target CPU needs RV64 machine instructions.

The **toolchain** is the set of cooperating build tools. The compiler and assembler translate source into target machine code. They produce **object files**: target-specific intermediate files that still need to be combined. The **linker** combines those objects and resolves references between them.

These build products use **ELF (Executable and Linkable Format)**, a binary format used here for both object files and the final executable.

A simplified build path is:

```
source code
    ↓
cross compiler / assembler
    ↓
object files
    ↓
linker
    ↓
RV64 executable
    ↓
QEMU
    ↓
RV64 target machine
```

A **cross compiler** runs on one architecture while producing code for another target architecture. This is **cross-compilation**.

QEMU has a different role: it does not translate source code into the executable. It provides the machine on which the already generated target program can run.

1.5.1 Object files are not yet the complete program

Compilation and assembly can already produce target-specific object files. These can contain RV64 machine code, but they are not yet the complete executable. Separate pieces still have to be combined and references between them resolved.

The final result of this build path is a linked RV64 ELF executable that QEMU can load for the target machine.

Lecture 2 unfolds this path manually: we inspect the object files, discover what is still unresolved, and then see why linking and the target memory layout are necessary.

1.6 When does an operating system become useful?

The `Hello BVS1` target can run one tiny program without a guest operating system.

Now change the requirement:

> **The machine must run two programs.**

New questions immediately appear:

- Which program receives the processor?
- Where is each program's state kept?
- What happens when one program waits for a device?
- Can one program overwrite another program's memory?
- Who coordinates access to shared devices?
- How does a program request a shared service?

Increase the requirements further:

- many programs,
- multiple users,
- persistent storage,
- keyboard and display,
- networking,
- failures,
- programs that do not trust one another.

The problem is no longer merely “execute instructions”.

Now the system needs mechanisms for **sharing, coordination, protection and abstraction**.

This gives us the working definition used in BVS1:

> An **operating system** is software that helps turn hardware into a usable computer system by providing abstractions, resource management, coordination, protection, and services for programs and users.

This is intentionally a working definition.

We will test and sharpen it as concrete mechanisms are introduced.

Two classical viewpoints are already visible:

1. **Operating system as resource manager**
   CPU time, memory, storage and devices are finite resources that must be shared and controlled.
2. **Operating system as abstract machine**
   Programs should not need to manipulate every hardware detail directly. The operating system can provide safer and more useful abstractions and services.

1.7 Applying the working definition

The opening lecture deliberately asks students to classify several familiar terms before the working definition is established.

After deriving the definition, the distinctions become clearer:

- **Windows, Ubuntu and Android** are complete operating-system environments.
- The **Linux kernel** is the core used by Linux-based operating-system environments, not such an environment by itself.
- **UEFI** is firmware and a firmware interface/environment.
- **QEMU** is an emulator and virtualizer.
- **Docker** provides container tooling built on operating-system mechanisms.
- **Firefox** is an application.

The purpose of the exercise is not to memorize labels.

It is to ask which responsibilities a component actually assumes and at which layer it operates.

1.8 From Hello BVS1 to a minimal operating system

The course deliberately starts with a target that can do very little:

```
Hello BVS1
```

The semester question is:

> **What must we add to Hello BVS1 before we would call the result an operating system?**

Initial answers may include:

- handling devices,
- multiple executions,
- scheduling,
- protection,
- memory management,
- system services,
- communication,
- persistent storage,
- users,
- a command interface.

At the beginning these are hypotheses, not a checklist to memorize.

We add mechanisms only when a new requirement gives us a reason for them.

The practical target remains:

> **a small educational operating system on RV64 and QEMU `virt`, eventually providing a simple shell**

The red thread is:

```
a machine executes code
    ↓
software communicates with devices
    ↓
events must be handled
    ↓
multiple programs must coexist
    ↓
programs compete for the CPU
    ↓
programs and resources need protection
    ↓
```

```
programs need operating-system services
    ↓
programs need coordination
    ↓
programs need communication
    ↓
a user needs to control programs
    ↓
a simple shell
```

The course is therefore not organized as a catalogue of terminology.

Each new mechanism should answer a problem that the current system has acquired.

1.9 Key ideas

Lecture 1 establishes a small number of distinctions that we will use repeatedly:

- A target processor can execute a program without a guest operating system.
- CPU, memory and devices exist independently of the OS.
- An ISA defines the machine-level contract on which software relies.
- Development host and target are different roles.
- Virtualization and emulation are different techniques.
- Multipass gives us the virtualized development environment; QEMU emulates our RV64 target in the BVS1 setup.
- A cross-toolchain produces machine code for a target ISA different from the development system.
- Operating systems become useful as requirements for sharing, abstraction, coordination, protection and services grow.
- The BVS1 system grows only when a new requirement motivates a new mechanism.

A useful test of understanding is to explain this complete path:

```
source code
    ↓
cross-toolchain
    ↓
RV64 executable
    ↓
QEMU program in the development VM
    ↓
emulated RV64 machine
    ↓
observable program behaviour
```

and then explain why successful execution still does **not** imply that the target contains an operating system.

Related Exercise

Exercise 1 — From Source to First Instructions also consolidates concepts from this lecture, especially the development-host/target distinction and virtualization versus emulation. Its main placement is after Lecture 2, where it also applies the toolchain and startup material.

References

- Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*. https://pages.cs.wisc.edu/~remzi/OSTEP/
- Andrew S. Tanenbaum and Herbert Bos, *Modern Operating Systems*, 5th ed., Pearson, 2023. ISBN 978-0-13-761888-0. https://www.pearson.com/en-us/subject-catalog/p/modern-operating-systems/P200000003295/9780137618880
- RISC-V International, *RISC-V Ratified Specifications Library*. https://docs.riscv.org/
- QEMU Project, *RISC-V `virt` Generic Virtual Platform*. https://www.qemu.org/docs/master/system/riscv/virt.html
- Canonical, *Multipass Documentation*. https://documentation.ubuntu.com/multipass/

2 From Source to Executables: Towards Processes

Lecture 1 showed that an RV64 target can execute a small program without a guest operating system. Lecture 2 asks how that execution comes into existence and why, for an operating system, simply starting a program is not enough.

The path is:

```
source → objects → linked ELF → memory → startup code → main
```

and then:

```
executing program → OS-managed execution → process
```

The build tools are therefore not the final topic. We unfold them once because they make the transition from source code to a running machine state visible.

2.1 From source code to target-specific object code

We first compile `main.c` without linking:

```
mkdir -p build/manual

riscv64-unknown-elf-gcc -march=rv64imac -mabi=lp64 \
  -mcmodel=medany -msmall-data-limit=0 \
  -c main.c \
  -o build/manual/main.o
```

The important option is `-c`: stop before linking. The remaining options are course-provided target/build settings.

Inspecting the result with

```
file build/manual/main.o
```

shows that `main.o` is already a RISC-V ELF object.

ELF (**Executable and Linkable Format**) is the binary file format we use here for both object files and the final executable.

This distinction matters: the compiler has already translated C into code for our target ISA. The object file is therefore not architecture-neutral. But it is also not yet a complete program. Separate pieces can still refer to one another, and their final addresses have not yet been established.

The startup assembly is translated in the same way:

```
riscv64-unknown-elf-gcc -march=rv64imac -mabi=lp64 \
  -mcmodel=medany -c start.S -o build/manual/start.o
```

2.2 Why object files still need a linker

A **symbol** is a name used to refer to a program entity or location, such as a function or a linker-defined address.

Using `nm` ("name list") lets us inspect the symbols in the two objects:

```
riscv64-unknown-elf-nm build/manual/start.o
riscv64-unknown-elf-nm build/manual/main.o
```

For the symbols relevant here, we can read the result as:

```
start.o defines _start
start.o needs   main
start.o needs   _stack_top

main.o  defines main
```

The important concept is not the letters printed by `nm`, but the relationship between separately translated pieces. One object can provide a symbol while another refers to it. The linker connects those pieces.

The unresolved `_stack_top` also reveals a second responsibility of the linker: it must establish where the program lives in target memory.

2.3 From the stored-program idea to a concrete memory layout

Lecture 1 used the abstract model:

```
instructions + data
        ↓
      memory
        ↕
       CPU
```

Our real program now contains different kinds of bytes that need concrete places.

The **stack** is runtime memory used during function calls, for example for local state and saved call information. The register `sp` is the stack pointer; `_stack_top` names the initial top address of this stack.

```
RV64 memory from 0x80000000

+-----------------------------------------------------------+
| .text    _start, main, ...      | instructions          |
| .rodata  "Hello BVS1\n"         | constant data         |
| .data                           | initialized data      |
| .bss                            | zero-initialized data |
| C stack  (top: _stack_top)      | runtime data          |
+-----------------------------------------------------------+
```

These roles are not properties that memory itself knows. They arise from the executable layout and from how the program and CPU use the addresses.

The course-provided `linker.ld` makes that layout concrete:

```
ENTRY(_start)

MEMORY {
    RAM (rwx) : ORIGIN = 0x80000000, LENGTH = 128M
}

. = ALIGN(16);
. += 16K;
_stack_top = .;
```

Here `.` is the linker's current output address. After 16 KiB have been reserved for the initial stack, that address is named `_stack_top`.

Students do not need to write linker scripts in BVS1. The important idea is that the linker does not merely join files; it constructs one final arrangement of addresses in target memory in which the pieces can refer to one another.

2.4 Linking produces the program QEMU can load

We link the two objects using the course layout:

```
riscv64-unknown-elf-gcc -nostdlib -nostartfiles -no-pie \
  -T linker.ld build/manual/start.o build/manual/main.o \
  -o build/manual/hello-bvs1.elf
```

Here `-T linker.ld` selects our course-provided memory layout; the remaining options are link settings for our target without a guest operating system.

At this point the linker has:

- combined the objects;
- resolved the symbol references;
- established the final memory layout.

The resulting ELF is what QEMU receives. QEMU does not compile or link the program; it provides the emulated RV64 machine and loads the already-built image.

2.5 Why execution starts before `main`

The linker script declares:

```
ENTRY(_start)
```

and `_start` is defined in `start.S`:

```
_start:
    la   sp, _stack_top
    call main
1:
    j    1b
```

This small fragment exposes an important boundary between machine startup and ordinary C code.

Before `main` can run, startup code establishes at least the execution environment that C expects. Here that means setting `sp`, the stack pointer, to the address denoted by `_stack_top`.

Then `call main` transfers control to `main` and remembers where execution should return. We deliberately do not open the full function-call mechanism yet. The fact that a return point must somehow be preserved becomes important in Lecture 3.

The local label `1:` and `1b` only express the final fallback loop: if `main` returns, execution remains there.

2.6 The whole path, and what Make hides

Running the ELF manually:

```
qemu-system-riscv64 -machine virt -m 128M \
  -smp 1 -nographic -bios none \
  -kernel build/manual/hello-bvs1.elf
```

reconstructs the complete path:

```
main.c    → compile  → main.o  ──┐
                                 │
start.S   → assemble → start.o ──┼──→ link → hello-bvs1.elf
                                 │                 ↓
linker.ld ───────────────────────┘               QEMU
                                                  ↓
                                                 RV64
                                                  ↓
                                              Hello BVS1
```

Compilation, assembly, linking, and QEMU itself run in the Ubuntu development VM. The final RV64 instructions execute in the emulated target.

Make automates this workflow. It decides when a target is out of date and runs the corresponding recipe. The GCC driver performs the translation steps and invokes the linker. QEMU provides the machine on which the result executes.

Doing the path manually once is useful because the automation then has a visible meaning rather than being a black box.

2.7 Change, rebuild, observe

If only the string in `main.c` changes, we can predict the consequences before rebuilding:

- `main.o` must change;
- `start.o` does not;
- the final ELF must be linked again;
- `linker.ld` does not change;
- QEMU itself does not change.

These are the logical dependencies. A concrete Makefile may encode them at a coarser granularity and therefore rebuild more than the minimum set of intermediate files.

This small exercise is about maintaining a causal model of the system:

predict → change → build → run → observe → explain

That pattern will be reused when the changes are no longer only strings, but operating-system mechanisms.

2.8 Why an executing program is not yet a process

Our current target has one execution:

```
_start
  ↓
main
  ↓
print
  ↓
for (;;) {}
```

Once it starts, nothing inside our target can currently stop it, regain control, and later continue it.

That is the new requirement. An operating system must be able to manage an execution rather than merely start it.

For our first model:

> A **process** is an execution that the operating system can manage.

For Checkpoint 1, "manage" means only:

```
create → run → yield → resume → exit
```

This is intentionally much smaller than a complete Unix process model. We do not yet need address spaces, files, process IDs, or scheduling policies.

The minimum lifecycle is:

```
create ──→ READY ──run / resume──→ RUNNING ──exit──→ FINISHED
             ↑                        │
             └──────── yield ─────────┘
```

A yielded process is still runnable. A finished process is not.

2.9 Yield, exit, resume, and restart

Yield and exit both transfer control back to the operating system, but they have different meanings.

- **yield**: the execution is not finished and must be able to continue later;
- **exit**: the execution is finished and must never continue.

The distinction becomes visible in this intended interface:

```
uart_puts("A\n");
process_yield();
uart_puts("B\n");
process_exit();
```

`process_yield()` and `process_exit()` are not library functions we already have. They are functions our minimal operating system still has to implement.

After `process_yield()`, the operating system has control. When the process runs again, it must continue with `uart_puts("B\n")`. Printing `A` again would mean that we restarted the execution instead of resuming it.

This is the conceptual step that makes the next lecture necessary.

2.10 The question that leads to execution context

To resume correctly, something about the paused execution must survive:

```
process runs
     ↓
yield
     ↓
OS regains control
     ↓
resume
     ↓
continue exactly where it stopped
```

What must be preserved so this can work?

At this point it is enough to suspect that some machine state must survive the pause. Lecture 3 determines exactly what that state is.

The final path of Lecture 2 is therefore:

```
source → objects → linked ELF → memory → _start → main
```

followed by:

```
executing program → OS-managed execution → process
```

The remaining question is:

> **What machine state makes resume possible?**

2.11 Key ideas

Lecture 2 connects the build path to the first process abstraction:

- Compilation and assembly produce target-specific object files.
- ELF is the binary format we use for both object files and the final executable.
- Symbols name program entities or locations; the linker resolves references between separately translated pieces.
- The linker also establishes the final memory layout of the program.
- The stack is runtime memory used during function calls, and `sp` is the stack pointer.
- Execution begins in startup code at `_start`, which establishes the initial environment before entering C.
- QEMU loads the already-built ELF into the emulated RV64 target; it does not compile or link the program.
- Make automates the build workflow, while the compiler, assembler, and linker perform the translation steps.
- A process is an execution that the operating system can manage through `create → run → yield → resume → exit`.
- `yield` and `exit` both return control to the operating system, but only a yielded execution may continue later.
- Resuming means continuing where execution stopped, not restarting from the beginning.

A useful test of understanding is to explain both chains:

```
source → object files → linked ELF → memory → _start → main
```

and

```
executing program → OS-managed execution → process
```

and then explain why resuming a process requires some machine state to survive the pause.

Related Exercise

Exercise 1 — From Source to First Instructions is the exercise associated with this lecture. It reconstructs the host/target layers, follows the source-to-executable path, inspects symbols and startup code, and prepares the transition toward the first process.

References

- RISC-V International, *RISC-V ABIs Specification*. https://riscv-non-isa.github.io/riscv-elf-psabi-doc/
- GNU Compiler Collection, *RISC-V Options*. https://gcc.gnu.org/onlinedocs/gcc/RISC-V-Options.html
- GNU Binutils, *The GNU linker (ld)*, especially the sections on linker scripts, symbols, `MEMORY`, and `SECTIONS`. https://sourceware.org/binutils/docs/ld.html
- GNU Binutils, *nm*. https://sourceware.org/binutils/docs/binutils/nm.html
- QEMU documentation, *RISC-V System Emulator*. https://www.qemu.org/docs/master/system/target-riscv.html
- QEMU documentation, *`virt` Generic Virtual Platform*. https://www.qemu.org/docs/master/system/riscv/virt.html
- Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, especially the process chapters. https://pages.cs.wisc.edu/~remzi/OSTEP/

3 First Process: Stop and Continue

Lecture 2 ended with a requirement: run an execution, regain control, and continue that same execution later. Two questions follow: **what must survive the pause, and how do we build that?**

Our working definition is:

> **A process is an execution that can be managed.**

In BVS1, our minimal operating system will do the managing. For our first process, managing means:

```
create -> run -> yield -> resume -> exit
```

This is the simplified lifecycle used for Checkpoint 1, rather than a general definition of every process's lifetime. To stop and continue the same execution, our process needs an entry function, its own stack, saved CPU register state (its context), and lifecycle information: state and exit status.

This chapter builds those parts into one resumable execution. Creation prepares it without running its code; run starts or resumes it; yield returns control while preserving the ability to continue; exit finishes it.

3.1 C pointer notation: a reminder

The process representation will be a C struct. These expressions distinguish an object, its address, and a pointer to it:

```
struct process p;
struct process *q = &p;
```

Here `p` is the object, `&p` is its address, and `q` stores that address. In the declaration, `*` makes `q` a pointer; in an expression, `*q` dereferences the pointer to access the object.

`p.state` accesses the object's state directly. `(*q).state` accesses the same member through the pointer; `q->state` is shorthand for that expression. The same notation lets us access a nested member such as `q->context.sp` once the context has been added.

For function pointers, storing a function and calling it are different operations:

```
p.entry = worker;
p.entry();
```

In the assignment, `worker` converts to a pointer to that function, so the first line stores where to call. It does not call `worker`. The second line calls it through the stored pointer; `worker()` would call the function directly.

3.2 Building the process object

We first give the execution a concrete representation. The following struct fragments add one part at a time; the complete definition follows after we establish what must be saved in a context.

3.2.1 Entry function

An entry function is where a new process begins its work. For example:

```
void worker(void) { /* process-specific work */ }
```

The process must remember which function to enter. Its first field is therefore a function pointer:

```
struct process {
    void (*entry)(void);
};
```

Read `void (*entry)(void)` as: `entry` is a pointer to a function taking no arguments and returning `void`. Repeating that declaration is awkward, so a `typedef` gives the pointer type a name:

```
typedef void (*process_entry_t)(void);
```

The field can now be written `process_entry_t entry;`. This names the same function-pointer type; it does not declare or call an entry function.

3.2.2 Owned stack storage

The process also reserves memory for its own call stack:

```c
#include <stdint.h>

#define STACK_SIZE 4096

struct process {
    process_entry_t entry;
    uint8_t stack[STACK_SIZE];
};
```

`uint8_t`, defined by `<stdint.h>`, is an unsigned 8-bit integer type. A `char stack[STACK_SIZE]` array would also be a viable choice for raw byte storage in our RV64 environment: a character array need not contain text. We use `uint8_t` to make the byte-storage role explicit and keep our treatment of integer types focused on `<stdint.h>`. This header is available in our freestanding RISC-V toolchain; including it does not require linking a C library.

At this point, treat the array simply as memory owned by the process; we will explain its use below.

3.2.3 Lifecycle information

The manager needs to know whether the process may run and, once it finishes, which exit status it produced:

```c
enum process_state { READY, RUNNING, FINISHED };

struct process {
    process_entry_t entry;
    uint8_t stack[STACK_SIZE];
    enum process_state state;
    int exit_status;
};
```

An `enum` supplies named alternatives for the state. READY means the process may run, RUNNING means it is executing, and FINISHED means it must not run again. `exit_status` records the result once it finishes.

3.2.4 What is still missing?

We can now remember where to start, where call data can live, whether the process may run, and how it finished. But suppose execution pauses here:

```
helper();
process_yield();
/* continue here later */
```

These fields do not yet tell us how to continue at exactly that point. We need to understand both the stack's runtime data and the CPU register values that make that data usable.

3.3 Understanding the process stack

The process needs **owned stack storage**: memory that remains associated with the process while the operating system executes elsewhere.

The process's machine instructions are **not** copied onto that stack. They remain in the program's `.text` section. The stack contains runtime data associated with active calls—for example saved register values, return information, some local variables, and temporary values when needed. Not every local variable is stored on the stack; the compiler may keep values in registers.

The array reserves storage; `sp`, the CPU's stack-pointer register, selects the current position in that storage. Array indices increase toward higher addresses, while the RV64 C stack grows toward lower addresses:

```
                  stack grows this way
                        <-----

lower addresses                            higher addresses
&stack[0]               sp                &stack[STACK_SIZE]
    v                   v                             v
    +---------------------------------------------+
    |                   | -- used stack ------------|
    +---------------------------------------------+
```

`&stack[STACK_SIZE]` is the address just past the array's last byte. A fresh process starts with an appropriately aligned `sp` at the high end; stack use then extends toward lower addresses. Reserving the array alone does not change the CPU's active stack pointer.

Nested calls belong to the same execution and use the same process stack. For example:

```
void helper(int *x) { (*x)++; }
void worker(void) { int n = 1; helper(&n); }
```

Conceptually, while `helper()` is active:

```
                          sp
lower addresses           v               higher addresses
    +-----------------------------------------------+
    |             | helper frame | worker frame | - |
    +-----------------------------------------------+
```

The `helper frame` and `worker frame` labels denote runtime data associated with the active calls. The stack does **not** contain the machine instructions of `helper()` or `worker()`: the CPU executes those instructions from the `.text` section of the program's memory layout.

The newer call uses space toward the lower-address end of the used stack. This is a schematic picture, not an exact compiler-generated layout: the compiler may inline calls or keep values in registers.

If the process yields from a nested call, that call chain's stack data must remain intact while the OS uses its own stack. Resuming requires the process's saved `sp` as well as the preserved storage.

3.4 Continuation: where should execution return?

Lecture 2 showed the startup fragment:

```
call main
```

A RISC-V `call` remembers the location to which execution should later return. That location is stored in `ra`, the **return-address register**.

An ordinary return therefore already contains a useful idea:

```
call function
    |
    v
function executes
    |
    v
ret
    |
    v
continue at address stored in ra
```

If a process yields and is later resumed, the operating system must preserve some representation of the place where execution should continue. Preserving `ra` makes that idea concrete.

The stack data must survive too, and the process must recover the stack position recorded in `sp`. Restoring a continuation with the OS stack would not continue the same call chain.

3.5 Context

At a pause, some execution state is still in CPU registers. Another execution can overwrite those values even when the process stack remains intact. The saved CPU register state needed to continue the execution is its **context**.

For the cooperative CP1 mechanism, the provided representation is:

```
struct context {
    uintptr_t ra;    // continuation address
    uintptr_t sp;    // current stack position
    uintptr_t s[12]; // saved s0 through s11
};
```

`uintptr_t`, defined by `<stdint.h>`, is an **unsigned integer type**, not a pointer type. It can hold the integer representation of a pointer value and later be converted back without losing that value. In our RV64 toolchain it is 64 bits wide. We use it for saved register-sized values such as `ra` and `sp`, which contain addresses during normal execution. Students are not expected to study the complete family of fixed-width integer types here.

The array `s` holds the RISC-V callee-saved registers `s0` through `s11`: `s[0]` stores `s0`, through `s[11]` storing `s11`. A **calling convention** is the agreement that lets separately compiled functions cooperate about how registers are used. A **callee-saved register** is one whose value a called function must preserve for its caller. Preserving those required registers allows an ordinary C function call to cross the context-switch boundary safely.

Students do not need to memorize that register list or derive the complete assembly routine from the ABI. For CP1 they should be able to explain why `ra` and `sp` matter and why some additional register state must also survive.

3.6 The complete process object

The saved context supplies the missing part. Together, entry, stack, context, and lifecycle information represent our managed execution.

The public representation used by Lecture 3, Exercise 2, and Checkpoint 1 is:

```
struct process {
    process_entry_t entry;
    uint8_t stack[STACK_SIZE];
    struct context context;
    enum process_state state;
    int exit_status;
};
```

The five fields have distinct roles:

- `entry`: where a new process begins;
- `stack`: process-owned raw byte storage for its call stack;
- `context`: saved CPU register state needed to continue execution;
- `state`: whether the process is READY, RUNNING, or FINISHED;
- `exit_status`: the result recorded when the process finishes.

Exercise 2 uses this exact field order because its precompiled process library was built against this representation. Checkpoint 1 keeps the same interface and representation while students provide the implementation.

3.7 The provided context-switch boundary

The course provides a function implemented in RISC-V assembly, with this C declaration:

```
void context_switch(
    struct context *current_context,
    const struct context *next_context);
```

`current_context` points to storage where the running execution's registers will be saved. `next_context` points to the saved registers to restore. The `const` qualifier says the function does not modify the context object through `next_context`. Either context may belong to the OS execution or the process execution.

A **context switch** in this part of BVS1 means:

1. save the current execution context;
2. restore another saved context;
3. continue the restored execution.

The assembly below shows the matching save and restore operations. The ellipses stand for the corresponding operations on `s1` through `s10`:

```
sd ra,    0(a0)
sd sp,    8(a0)
sd s0,   16(a0)
...
sd s11, 104(a0)

ld ra,    0(a1)
ld sp,    8(a1)
ld s0,   16(a1)
...
ld s11, 104(a1)
```

```
ret
```

`sd` (store doubleword) copies a 64-bit register value into memory; `ld` (load doubleword) copies a 64-bit value from memory into a register. In `sd s0, 16(a0)`, the destination is the address in `a0` plus 16 bytes. The offsets follow the context layout: `ra` at byte 0, `sp` at byte 8, and the twelve saved `s` registers at bytes 16 through 104.

After the loads, `ret` uses the newly restored `ra`. The switch therefore changes both the stack that is active and the location at which execution continues.

The stored fields and the restored fields match: `ra`, `sp`, and all twelve entries of `s`. The process needs both its stack memory and these saved register values to continue the same execution.

The function arguments `current_context` and `next_context` arrive in the RISC-V argument registers `a0` and `a1`. These registers are caller-saved: their previous values are not part of the state that an ordinary C call promises to preserve. In this cooperative switch they are used to locate the two context structures; the persistent saved context is `ra`, `sp`, and `s0-s11`.

The same operation works in both directions:

```
OS running                          process running
    |                                    |
    | save OS context                    | save process context
    | restore process context            | restore OS context
    v                                    v
process running                       OS running
```

There is no scheduler here. CP1 contains only one process.

3.8 Implementation boundary

The course provides the machinery whose complexity is incidental to the CP1 competence:

- existing startup and linker support;
- UART output already available in the starter;
- the context representation;
- the RV64 context-switch assembly boundary.

Students construct the process abstraction and lifecycle in C:

- process representation;
- owned stack;
- first-run context;

- entry handling;
- run;
- yield;
- exit;
- observable resume behaviour.

CP1 does **not** require timer interrupts, preemption, several runnable processes, a scheduler, privilege separation, virtual memory, dynamic loading, or dynamic allocation.

3.9 Public process interface

The first-process abstraction uses this public C interface:

```
void process_create(struct process *process,
                    process_entry_t entry);
void process_run(struct process *process);
void process_yield(void);
void process_exit(int status);
```

Exercise 2 links this interface against a provided implementation. Checkpoint 1 keeps the same interface but replaces that implementation with student-written C code.

3.10 The lifecycle in C

Lecture 2 introduced:

```
create -> run -> yield -> resume -> exit
```

Lecture 3 now connects each semantic step to the minimum implementation work.

3.10.1 Create / prepare

Preparing a process means constructing a valid first-run state:

```
remember entry function
        |
choose process stack top
        |
prepare initial context
        |
state = READY
```

No process code has executed yet.

A process that has already yielded has a saved continuation. A new process does not.

The restore mechanism nevertheless expects a context containing a stack pointer and a continuation. The solution is to prepare an initial context before the process first runs.

Its initial stack pointer is placed at the aligned high end of its owned stack array.

The initial continuation is a small **bootstrap function**, called `process_start()`: an internal function used to enter a process for the first time. It is an implementation detail, not another public process operation.

Conceptually:

```
initial sp -> aligned high end of process.stack
initial ra -> process_start -> process.entry()
```

The first restore followed by `ret` therefore enters `process_start()` while using the process's own stack.

`process_start()` calls the process entry function. If that function returns normally, `process_start()` calls `process_exit(0)`. Returning from the entry function therefore finishes the process with status 0 and transfers control back to the OS.

Later resumes do not use the prepared bootstrap address again. A yield saves the real continuation from the running process, so the next restore continues after the context-switch call inside the yield path.

This is an important simplification:

First start and later resume use the same low-level restore mechanism.

3.10.2 Run and resume

The OS keeps a pointer named `current_process` to identify the process whose execution currently owns the CPU. `process_yield()` and `process_exit()` take no process argument: they use this pointer to find the object whose state and context they must update.

`process_run(p)` performs these steps:

```
reject p if its state is FINISHED
set current_process to p
set p's state to RUNNING
context switch from OS to p
```

The FINISHED check belongs to `process_run()`: a saved context does not by itself grant permission to run again.

The same run operation can support two cases:

- a READY process that has never run starts through its prepared bootstrap context;
- a READY process that yielded earlier resumes through the context saved by that yield.

The required behaviour is **resume**. A separate `process_resume()` API is not necessary.

3.10.3 Yield

For CP1:

> **yield** means that the running process voluntarily returns control to the operating system while remaining able to resume later.

Using `current_process`, yield marks the process READY, saves its current context, and restores the OS context.

When the operating system later runs that process again, execution continues after the context-switch call inside the yield path.

3.10.4 Exit

For CP1:

> **exit** means that the process finishes, records a result, and returns control to the operating system.

Using `current_process`, exit records the status in the process object, marks it FINISHED, and switches from the process context back to the OS context.

A finished process must not be resumed.

Yield and exit can use the same low-level control-transfer boundary, but their lifecycle meaning is different.

3.11 The complete CP1 observation

Exercise 2 and Checkpoint 1 use the same small permutation program. Its observable behaviour is:

```
process: before yield: VBS1
os: process yielded
process: result: VB1S
os: process exited
```

The interesting evidence is not the strings themselves. The output demonstrates that:

1. the process starts using its own execution context;
2. `worker()` calls `permute()`, which modifies process-local data and yields before returning;
3. the operating system regains control while that call chain remains suspended;
4. the same `permute()` invocation later continues after `process_yield()`;
5. the local array progresses from `VBS1` to `VB1S`, showing that the same execution state survived;
6. the process exits with status 0 and becomes FINISHED;
7. the operating system regains control again and must not resume the finished process.

3.12 Why CP1 deliberately uses yield

CP1 needs a control transfer that gives the operating system control **without finishing the process**.

A timer interrupt would already introduce the preemption mechanism needed later for CP2. Blocking or sleeping would require an additional condition that makes the process runnable again. An ordinary C return would not expose the requirement to preserve and later restore a continuation.

A cooperative yield is therefore the smallest mechanism that makes resumability observable.

It also creates the next operating-system problem cleanly:

> What if a process never yields?

That question motivates timer-driven preemption on the way to CP2.

3.13 Check your model

Use these questions to check the distinction between restarting, resuming, and finishing:

1. What breaks if we call the entry function again after yield instead of restoring the saved context?
2. What breaks if we restore `ra` but not `sp` and `s0-s11`?
3. What breaks if `process_run()` allows a FINISHED process to execute again?

A successful explanation should connect each failure to the process representation: calling entry again restarts its work, restoring only part of the context cannot preserve the same execution, and restoring a finished process violates its lifecycle state.

3.14 Key ideas

- A **process** is a managed execution.
- Our process representation combines an entry function, owned stack storage, CPU context, lifecycle state, and exit status.
- The **stack** is process-owned memory for the runtime data of active function calls. Function instructions remain in `.text`.
- A **context** is the saved CPU register state needed to continue the same execution.
- **Lifecycle state** says whether an execution may run; **context** says how it continues.
- A **context switch** saves the current CPU register state and restores the next one. The same mechanism transfers control in both directions between OS and process.

Related Exercise and Checkpoint

Exercise 2 — Using Your First Process lets you use the process abstraction before implementing it yourself.

Checkpoint 1 — I Have a Process is the corresponding implementation milestone.

References

- RISC-V International, *RISC-V ABIs Specification*. https://riscv-non-isa.github.io/riscv-elf-psabi-doc/
- Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, especially the process chapters. https://pages.cs.wisc.edu/~remzi/OSTEP/
- Russ Cox, Frans Kaashoek, Robert Morris et al., *xv6: a simple, Unix-like teaching operating system* and xv6-riscv source, used here only as a comparative reference for low-level context switching. https://pdos.csail.mit.edu/6.S081/2020/xv6/book-riscv-rev1.pdf

4 Where Our Code Runs

Lecture 3 constructed a process from an entry function, owned stack storage, saved CPU context, and lifecycle information. This chapter connects that abstraction to the machines and memory we actually use. It follows four questions raised by students: Where am I working? How do I edit the files that are actually built? Where is the program counter? Where do constants appear in the disassembly? Together they lead to the guiding question: **where are our instructions and runtime data, and what must be preserved when execution pauses or is interrupted?**

We use the same `worker()` and `permute()` from Exercise 2. The aim is to reconstruct and diagnose the existing mechanism before extending it.

4.1 Locate editing, building, and execution

Our setup has three machine roles:

```
physical host: Linux / macOS / Windows
    |  hosts the VM; Multipass manages it
    v
Ubuntu VM bvs1: our development host
    |  runs editor, build tools, and QEMU
    v
QEMU emulates an RV64 target: CPU + RAM + UART
    |  target CPU executes RV64 instructions
    v
our program / growing BVS1 operating system
```

The physical host supplies the outer infrastructure. Multipass manages the Ubuntu VM; it is not another machine layer. The VM is the **development host** for our tools. QEMU runs there as an ordinary Ubuntu process and emulates the **target machine**.

Both our OS-side control code and the managed process execute on the RV64 target. They do not become Ubuntu programs merely because QEMU runs in Ubuntu. The cross compiler and linker execute in the VM but generate an RV64 ELF for the target ISA.

The complete workflow is:

```
edit -> save source -> compile/link -> linked ELF
                                            |
                                    QEMU loads the image
                                            |
                                   RV64 instructions execute
```

Make automates the build recipe. The editor changes source text; the compiler reads the saved file; QEMU receives the resulting ELF. An already running QEMU instance does not automatically acquire a new image when source files are edited or rebuilt.

4.2 Establish a reliable edit–build–run loop

Start on the physical host with:

```
multipass shell bvs1
```

Inside the VM, locate the existing repository and source file:

```
cd ~/teaching-bvs1-code
pwd
ls
nano main.c
```

Run `make` from the repository directory that contains the `Makefile`. Make reads the `Makefile` in the current directory, and that `Makefile` names the source files it compiles. A `main.c` edited in another directory or another copy of the repository is not this build's input.

In nano, save with **Ctrl-O**, **Enter**, then leave with **Ctrl-X**. If using Vim, enter Insert mode with **i**, make the change, press **Esc**, then save and leave with `:wq`. The editor handout contains further details. The relevant competence is editing the intended file reliably; the choice of editor is not assessed.

After saving, inspect your change and build:

```
git diff -- main.c
make
make run
```

Use one small output-label change to connect the saved file to an observable effect. The array's permutation should remain unchanged if only a label changes. End QEMU with **Ctrl-a**, release, then **x**. Restore that intentional label change before continuing.

If an old label appears, diagnose the chain in order:

```
saved source -> build input -> successful build -> executable -> QEMU run
```

Was the change saved? Is the saved file the one Make uses as build input? Did the build succeed and produce a new executable? Did a new QEMU session load that executable? Rebuilding repeatedly cannot correct an unsaved buffer or a different repository copy.

4.3 One managed execution can contain several active calls

Exercise 2 uses this entry function:

```
static void worker(void)
{
    char value[] = "BVS1";
    permute(value);
    uart_puts("process: result: ");
    uart_puts(value);
    uart_puts("\n");
    process_exit(0);
}
```

The local array has five bytes: `B`, `V`, `S`, `1`, and the terminating zero byte. It is modifiable character data. The helper receives a pointer to that existing array; passing it does not create another array or another process.

The helper first exchanges positions 0 and 1, prints the intermediate value, calls `process_yield()`, then exchanges positions 2 and 3. Before yield, both `worker()` and `permute()` are still active. The array has become `VBS1\0`, but its owning worker invocation has not ended.

4.4 Instructions and runtime data have different roles

The linked program places machine instructions in `.text`. This includes instructions implementing the worker, helper, process API, and OS-side code. Calling a function transfers control to its instructions; it does not copy those instructions onto the stack.

The **program counter**, `pc`, identifies the instruction to execute. The **stack pointer**, `sp`, selects the current stack position. These are CPU state, not memory sections. A saved return address is an address value that points back into code; its being stored on the stack does not make the pointed-to instructions stack contents.

The stack holds runtime data associated with active calls: saved call information, saved registers, some local variables, and temporaries as needed. The call data of one active call

is its **stack frame**. The labels `worker stack frame` and `permute stack frame` describe this runtime data, not the functions' instructions.

A schematic snapshot after the first swap is:

```
.text                        process-owned stack storage
                             higher addresses
same worker instructions     +--------------------------+
same permute instructions    | worker: value V B S 1 \0 |
  ...                        +--------------------------+
  call process_yield         | permute call data        |
                             +--------------------------+
                             lower addresses
```

The CPU has changed data in the array while executing instructions that remain in `.text`. A non-inlined helper call uses the same process stack as the worker, with newer call data toward lower addresses. The picture does not prescribe exact offsets, compiler-generated stack frames, or a stack slot for every local value.

4.5 `sp`, `ra`, and `pc` are different kinds of CPU state

`sp` and `ra` are the conventional ABI names of the integer registers `x2` and `x1`. Ordinary instructions read and write them like the other general-purpose registers `x0`–`x31`. The program counter is also architectural CPU state, but it is not one of these 32 registers. The CPU advances it as it executes, and jumps, branches, calls, and returns change it.

This explains why the Lecture 3 context contains `ra`, `sp`, and `s0`–`s11` but no explicit `pc`. The cooperative context switch happens at an ordinary function-call boundary, so the saved `ra` already describes the continuation:

```
saved ra -> restored ra -> ret -> execution continues there
```

`ret` sets the program counter to the restored `ra`. The program counter is therefore not forgotten; the cooperative switch re-establishes it through `ra`.

Note the distinction this relies on. No saved context contains the live `pc`, which changes with every instruction. It contains a **continuation address**: a saved value saying where the execution continues when it is restored. In CP1 that value is `ra`, and `ret` turns it back into `pc`.

4.6 Follow the same execution through yield and resume

When the process yields, the low-level switch saves its continuation, stack position, and required additional registers. The OS context is restored. The CPU's current `pc` and `sp` now describe OS execution, while `p.context` holds the saved register values for the paused process. The array and suspended call data remain in `p.stack`.

The next `process_run()` restores that saved context. Low-level continuation first returns through the suspended yield path. `process_yield()` then returns to the same `permute()` invocation, which performs the second swap. The array becomes `VB1S\0`. The worker does not initialize it again, because no new worker invocation was started.

When the helper returns, the worker prints the result and calls `process_exit(0)`. Exit marks the process FINISHED and returns control to the OS. Old bytes may physically remain after a helper returns or a process exits; that does not keep a call active or authorize another run. Lifecycle state determines whether the process may run again.

Moment	Array contents	CPU executes
worker initialized it	`BVS1\0`	process
first swap completed	`VBS1\0`	process
yielded	`VBS1\0`, preserved	OS
second swap completed	`VB1S\0`	process
process exited	no active array	OS

Calling entry again restarts the work; restoring a continuation without the matching stack position loses the original call chain; allowing a FINISHED process to run violates its lifecycle. These are different failures of the same model.

4.7 Connect the other memory regions

The section names describe roles in the linked image:

Region	Role in our example
`.text`	RV64 machine instructions
`.rodata`	constant data, including output labels
`.data`	writable static/global objects with stored initial values, when present
`.bss`	zero-initialized static/global storage, including OS bookkeeping in the inspected library
OS stack	runtime data of OS-side calls
process stack storage	runtime data of the managed process's calls

The output label `"process: result: "` is constant data, normally placed in `.rodata`. The initialized local `value` array is a separate writable object. Its C initializer does not put that array in `.rodata`: the compiler can provide its initial bytes from constants in instructions or from constant storage. Inspect the actual build rather than assuming a separate `BVS1` string must be present in `.rodata`.

Constants therefore appear in the disassembly in two forms:

- A value small enough to encode inside an instruction appears as an **immediate** operand, for example in `li` or `addi`. Larger values may be built by more than one instruction. The four characters of `BVS1` can appear together as one 32-bit value: our target uses little-endian byte order, so storing `0x31535642` produces the bytes `0x42 0x56 0x53 0x31` at increasing addresses.
- In this build, constant output labels are stored in `.rodata`. Instructions compute a label's address, typically with an `auipc`/`addi` pair that `objdump -d` annotates with the resulting address, and pass it to `uart_puts()`. `riscv64-unknown-elf-objdump -s -j .rodata build/hello-bvs1.elf` shows the bytes stored at that address.

For comparison, a static/global `int initialized = 7` typically belongs to `.data`; a static/global `int zeroed` initially has value zero and typically belongs to `.bss`. Static storage lasts for the execution of the program, unlike the local state of a particular call. Neither declaration is needed in the Exercise 2 application merely to fill a diagram.

In the inspected Exercise 2 image, `.data` is empty. The local `value` array belongs to neither section: it lives on the process stack while `worker()` is active. Its `.bss` contains `os_context` and `current_process` from the provided library. A pointer becoming nonzero at runtime does not move its storage to `.data`. BSS occupies memory even though the ELF does not store all its initial zero bytes as file contents; loading/startup must establish the required initial state.

These roles do not imply that our minimal system enforces read/write permissions or separate address spaces. The course linker script and runtime storage choices determine the layout we are studying.

4.8 The process stack is an array in an object

Exercise 2 declares:

```
void main(void)
{
    struct process p;
    /* create, run, inspect state, run again */
    /* ... */
}
```

Here `p` is an automatic object in `main()`'s OS-stack allocation. The embedded array `p.stack` is part of that object. It reserves a subregion that becomes the process's active call stack when `sp` selects an aligned position inside it.

This gives the process distinct stack storage for its execution without creating another ELF section. While the process runs, OS-side calls are suspended. When the OS regains control, it uses other stack space and leaves `p.stack` intact. `main()` remains active throughout, so the process object and its embedded storage remain alive.

If `p` were instead static/global, its containing storage would be static. The role of its stack array would remain process-owned runtime storage. Therefore "process stack" answers what the memory is used for; the declaration and allocation answer where that memory comes from. In this example no dynamic allocation is required.

4.9 Inspect the file without confusing it with live memory

After a successful Exercise 2 build, run these commands in the VM:

```
riscv64-unknown-elf-objdump -h build/hello-bvs1.elf
riscv64-unknown-elf-nm -n build/hello-bvs1.elf
riscv64-unknown-elf-objdump -d \
  --disassemble=context_switch build/hello-bvs1.elf
```

The first command lists sections; the second lists symbols by address; the third decodes the selected function's instructions. In `nm`, `T/t` identifies code symbols and `B/b` identifies BSS-related symbols. A symbol can simply name an address: `_stack_top` is the linker-defined initial OS-stack address, not a C array declaration. These inspection options are documented in the GNU Binutils manuals.

The commands inspect the ELF on disk. They do not display the running target's CPU registers or the live contents of the local array. Debugging information can describe local variables, but that is different from a linker symbol or an observation of runtime memory.

The normal build uses `-O2`. It can inline `permute()`, meaning it incorporates the helper's work into the caller instead of retaining a separate call. Local scalar values such as `tmp` can be kept in registers. The lecture's separate observation build uses `-O0 -g -fno-inline` to retain inspectable function boundaries; `-g` adds debugging information. Exact stack frames and addresses depend on the build. See GCC's Optimize Options.

4.10 The next requirement: regain control without cooperation

Our current mechanism depends on the process reaching yield or exit. If it runs forever before either operation, our target-side OS continuation cannot run again. Ubuntu can still

manage the QEMU process, but that is control at another layer. Terminating QEMU ends the target session rather than implementing target-side process management.

An **interrupt** signals an event. When enabled, it transfers control to a configured OS handler, allowing the OS to regain control. A future **timer interrupt** can make control transfer possible without a voluntary yield. This enables **preemption**: the OS takes control without the process's cooperation. Scheduling then decides whether to continue the interrupted execution or select another runnable one. Interrupt handling and the scheduling decision are related but distinct steps.

This chapter raises that question without requiring an interrupt implementation for CP1. An arbitrary interruption is not an ordinary C call boundary. The interrupted continuation is not supplied by `ra` as it is at our cooperative call boundary; `ra` holds whatever value the interrupted code was using. Once control has transferred into OS entry code, software can no longer recover the interrupted program counter unless the hardware captured it at that moment. It must therefore be preserved explicitly, in dedicated trap state, together with the other registers the interrupted code may be using. The cooperative CP1 register subset is not by itself the full specification of such a mechanism.

4.11 Key ideas

- The Ubuntu VM is our development host; the emulated RV64 machine is the target that executes our program's instructions.
- Editing, saving, building, and starting a new run are separate steps in one observable workflow.
- Function instructions remain in `.text`; stack frames contain runtime data associated with active calls.
- A local writable array is distinct from constant data used to initialize it or print an output label.
- Process stack storage is an array in the process object; its role does not prescribe an ELF section or an address space.
- Preserved stack data and saved register context together support continuing the same execution.
- The program counter is CPU state but not a general-purpose register: the cooperative switch re-establishes it through a saved continuation address (`ra` and `ret`), whereas after an interruption only the hardware can record where to continue.
- Lifecycle state determines whether another run is permitted, even if old memory contents remain.

Related Exercise and Handouts

Exercise 3 — Tracing Code and Runtime State investigates the workflow and memory roles using the existing Exercise 2 application.

The editor handout and development-environment handout support everyday work in the VM.

References

- GNU Binutils, objdump and nm.
- GCC, Optimize Options.
- RISC-V International, RISC-V ABIs Specification.
- Arpaci-Dusseau and Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*.

5 Entering the OS

Lecture 4 ended with a process that never yields. Our CP1 operating system regains control only when the process calls `process_yield()` or `process_exit()`; a process stuck in `for (;;)` never makes that call. This chapter answers the first half of the problem: **how can the CPU transfer control to the OS?** The answer is the trap. Lecture 6 then uses the same mechanism for an event the process did not ask for.

We keep the Exercise 2 application. Its `worker()` and `permute()` do not change. What changes is everything underneath: how control reaches the OS, what is saved, and what role `main()` plays.

5.1 The mental model of our current OS

5.1.1 The lifetime from Lecture 3, revisited

Lecture 3 drew the lifetime of our first process as two lines: the OS on the left, the process on the right, and context switches between them. There we drew only what we wanted the OS to be: absent while the process runs. Our construction did more. `main()` was itself a suspended execution: it was paused inside `process_run()` and `context_switch()`, its call chain stayed on the OS stack, and its registers were saved in `os_context`. Each yield resumed `main()` exactly where it had stopped. In the completed diagram, `!` marks this suspended OS execution (red on the slides):

```
       OS                          Process
       |
create |
   run +------- context switch ------->| (start)
       !                                |
       |<------ context switch --------+ yield
       |
   run +------- context switch ------->| (resume)
       !                                |
       |<------ context switch --------+ exit
       |
```

In CP1 the OS was managed exactly like a process: two executions taking turns. This is what "`main()` simulated the OS" means. The red line is the part of the picture this chapter removes.

5.1.2 What if the process never yields?

```
for (;;) { /* keep computing */ }
```

The CP1 OS gets control back only through the call to `context_switch()` inside `process_yield()` or `process_exit()`. Both are ordinary function calls made by the process. If the process never calls, our OS never runs again.

5.2 From calls to traps

5.2.1 Two halves of the answer

The full answer to a never-yielding process has two halves:

```
Lecture 5: the process asks       ecall   -> CPU -> OS
Lecture 6: an event interrupts    timer   -> CPU -> OS
```

Both use **the same entry path into the OS**. This lecture builds it for the deliberate case: the process still decides when it enters the OS, now through an instruction instead of a function call, so the trace is predictable. `process_yield()` and `process_exit()` keep their interface; only the mechanism behind them changes. Lecture 6 reuses the identical path for an asynchronous event; the only new part there is the source of the event.

The goal of the refactoring is that the OS really discontinues while the process runs: no more red lines.

5.2.2 The lifetime through the CPU

With traps, the same lifetime looks different:

```
         OS                CPU                   Process
                        +--------+
start-up | prepare      |        |
         +------------->| return |------------->| (start)
                        |        |              | ecall "yield"
         |<-------------|  trap  |<-------------+
 handler | decide       |        |
         +------------->| return |------------->| (resume)
                        |        |              | ecall "exit"
         |<-------------|  trap  |<-------------+
 handler | decide       |        |
```

The CPU is drawn as a box, not as a lifeline: it is not a third execution taking turns with the OS and the process, but the mechanism built into instruction execution that both rely on. It contains two mechanisms. **Trap** is the way into the OS: instead of continuing with the next instruction, the CPU records why and where, and continues at an address the OS chose earlier. **Return** is the way out: the OS executes a return instruction, and the CPU continues the selected process. The box is open at the bottom because every later trap and return goes through the same mechanism.

The red lines are gone. While the process runs, nothing of the OS is waiting; the OS runs only when the CPU traps into it, and after the return it is finished, not paused. Each OS bar is a short episode: start-up once, then one handler run per trap. The process is now the only long-lived execution. This does not mean that the OS forgets everything: its *data*, such as the process table, stays in memory. What disappears is a suspended *execution* with its own registers and call chain, and that is exactly what the red line was.

Even the first start is a return: the OS prepares saved state for the new process as if it had trapped into the OS just before its first instruction, and returns into it. Starting and resuming a process are the same operation for the CPU. The rest of the chapter replaces each plain word in this picture with code: "start-up", "prepare", "handler" and "decide".

5.2.3 What the CPU provides for traps and returns

The mechanism is the CPU's side of a contract, the ISA from Lecture 1. It consists of three kinds of parts:

Kind	Provided by the CPU	Used how
state	`mtvec`, `mepc`, `mcause`, `mstatus`, `mscratch`	read/written by the OS
instructions	`csrr`, `csrw`, `csrs`, `csrrw`, `mret`	executed by the OS
	`ecall`	executed by the process
rules	on a trap: record, interrupts off, jump	automatic, on an event
	on a return: restore interrupt state, continue	when the OS runs `mret`

The state lives in **control and status registers (CSRs)**: CPU registers outside the 32 general-purpose registers `x0`–`x31`. Ordinary loads and stores cannot reach them; only the CSR instructions of the `Zicsr` extension can, which is why our build uses `-march=rv64imac_zicsr`. In our code we use four assembler forms: `csrr` (CSR read) copies a CSR into a register, `csrw` (CSR write) copies a register into a CSR, `csrs` (CSR set) sets bits in a CSR, and `csrrw` (CSR read and write) does both at once. Only `csrrw` is itself one of the `Zicsr` instructions; the other three are convenient pseudoinstructions that the assembler translates into them.

`mret` (machine return) and `ecall` (environment call, a call into the execution environment, here the OS) are not CSR instructions: they read or write CSRs only implicitly, through the CPU's rules. `mret` applies the return rule; `ecall` causes a trap on purpose.

The OS configures the state, uses the instructions, and relies on the rules. The third row is why "services" would be a misleading word for what the hardware provides: a service is requested, whereas the trap rule is applied whether anyone asks or not. The CPU is also distinct from the **platform**, the machine around it: memory map, UART, and in Lecture 6 the timer. There a platform device raises the event, and the CPU's trap rule reacts to it.

5.3 Trap and return: what the CPU does

5.3.1 Why a function call is not enough

In CP1 the OS got control back only through a call the process made. A call happens only if the running code decides to make it, and it brings three things with it:

- the caller chooses the moment;
- the call instruction sets `ra`: where to continue;
- the calling convention says which registers may be lost.

This is why `context_switch()` could save only `ra`, `sp` and `s0`–`s11`. A process in `for (;;)` never calls, and an event between two instructions offers none of these properties. Each missing property becomes a job of the trap mechanism:

- no `ra`, so nobody says where to continue: the CPU records it;
- no chosen moment, so the event can arrive at any time: the CPU disables interrupts while the OS saves;
- no calling convention, so every register may be live: the OS saves all of them.

The first two jobs are the CPU's and follow in this section; the third is the OS's and follows in the next.

5.3.2 Traps

A **trap** is a transfer of control to an OS-defined handler that the CPU performs itself. "OS-defined" means that the OS chooses beforehand where the CPU continues; the CPU runs no OS code and makes no decisions, it only redirects the `pc`. RISC-V distinguishes two kinds:

	synchronous	asynchronous
caused by	the current instruction	an independent event
where	at that instruction, every run	between two instructions, varies
RISC-V name	exception	interrupt

	synchronous	asynchronous
"trap `pc`"	the causing instruction	the next instruction, not yet executed
in BVS1	`ecall`, illegal instruction (Lecture 5)	timer (Lecture 6)

A **synchronous** trap is caused by the instruction being executed. It is reproducible: it happens at the same instruction every time the program runs with the same input, and one can point to that line in the disassembly. An **asynchronous** trap is caused by an event outside the instruction stream; the CPU takes it between two instructions, wherever the program happens to be, so the place varies from run to run. Both kinds enter the same OS code, so "trap" is the umbrella term. Calling an `ecall` an interrupt blurs a distinction we need in Lecture 6.

This axis is separate from the question whether a trap is cooperative. An illegal instruction is synchronous but not cooperative: the process causes it but does not intend it as an entry into the OS. Today's normal case, `ecall`, is synchronous and cooperative; Lecture 6 changes both at once.

In this section, "handler" means all OS code that runs after a trap. The next section splits it into the provided assembly `trap_entry`, to which the CPU jumps, and the C function `trap_handler()`, which `trap_entry` calls.

5.3.3 Who can save the `pc`?

Lecture 4 showed that no saved context contains the live `pc`. It contains a **continuation address**: a saved value saying where an execution continues when it is restored. In CP1 the process supplied that address itself, because its call placed it in `ra`.

Suppose the CPU simply jumped into the OS, to the address the OS chose earlier:

```
process   ecall              <- must continue after this
OS        first instruction  <- live pc is here now
```

The reason this is a problem lies in what the `pc` is. The `x` registers and the CSRs are addressable: instructions name them as operands and read or write them. The `pc` (program counter) is not. It is **architectural state** that changes only implicitly, through control transfers (jumps, branches, `ret`, `mret`) and through the trap rule. No instruction stores "the pc" into memory the way `sd` stores a register.

Software is not blind to the program counter: PC-relative instructions such as `auipc` compute addresses relative to the current instruction. But at the first instruction of the OS code, the live `pc` describes the OS code. The OS can no longer recover where the process was, unless the CPU captured that address at the moment of the jump.

5.3.4 On a trap: record where and why

On a trap, the CPU first records **where** and **why**, in two CSRs:

CSR	Stands for	Holds
`mepc`	Machine Exception Program Counter	the **trap pc**
`mcause`	Machine Cause	bit 63: interrupt; bits 62..0: code

We use **trap pc** for the program-counter value that the CPU records in `mepc` on trap entry. This is our teaching term, not a named RISC-V architectural object. Despite "Exception" in its name, `mepc` is written on every trap, interrupts included. The meaning of the trap `pc` depends on the kind of trap:

- for an **exception**, it is the address of the instruction that caused it; for an `ecall`, the `ecall` itself;
- for an **interrupt**, it is the address of the instruction that was about to execute and has not executed yet.

The difference matters as soon as the OS returns: after an exception it may have to skip the causing instruction, after an interrupt it must not. Three terms now stay apart:

```
live pc               current execution position of the CPU
trap pc               value the CPU captured in mepc on trap entry
continuation address  saved value: where this execution resumes

interrupt:  continuation address = trap pc
ecall:      continuation address = trap pc + 4
```

`mepc` is a CSR, so software can read it; it holds a copy of a `pc` value captured by the CPU. Software never reads the `pc` itself.

The `mcause` values we need are:

`mcause`	Meaning	`mepc` holds
2	illegal instruction	that instruction
11	`ecall` from M-mode	the `ecall`
7, interrupt bit set	machine timer (Lecture 6)	the next instruction, not yet executed

On RV64, `mcause` is 64 bits wide: bit 63 is the interrupt bit, bits 62..0 hold the code. In hexadecimal the three values are `0x2`, `0xb` and `0x8000000000000007`. The same code can mean different things depending on the interrupt bit: exception 7 is a store/AMO access fault, interrupt 7 the machine timer.

Our model focuses on the minimal trap state we need: `mepc`, `mcause`, and the relevant `mstatus` fields. The RISC-V privileged architecture defines additional trap-related state, for example `mtval`, which we deliberately leave out.

5.3.5 On a trap: interrupts off

The CPU also changes two bits of the CSR `mstatus` (machine status), bits 3 and 7:

Bit	Stands for	Means
`MIE`	Machine Interrupt Enable	interrupts allowed (1) or not (0)
`MPIE`	Machine Previous Interrupt Enable	the value `MIE` had before the trap

```
MPIE <- MIE     remember whether interrupts were allowed
MIE  <- 0       no interrupt now
```

Nobody interrupts the OS while it saves the process's state. A second interrupt at that moment would overwrite `mepc` and `mcause` before they are saved. The precise rule is that the CPU saves `MIE` into `MPIE` and clears `MIE`; it is not a general statement that "traps disable interrupts". `MIE` does not affect exceptions: an `ecall` or an illegal instruction traps regardless. The CPU also records the privilege mode the trap came from in a third field, `MPP`; in our system that is always machine mode.

In Lecture 5 no interrupt is enabled yet, so the effect is not observable. Lecture 6 adds a second, similarly named CSR: `mie` (lower case) holds one enable bit per interrupt source. A timer interrupt is taken only if it is pending and both `mstatus.MIE` and the timer's bit in `mie` are set; Lecture 6 shows where "pending" is recorded.

5.3.6 On a trap: jump

```
pc <- mtvec        the address held in the CSR mtvec
```

`mtvec` (Machine Trap-Vector) holds a base address and a mode field. We use direct mode (mode 0), in which **every** trap goes to the **one** base address. `trap_entry` is 4-byte aligned, so its address leaves the two mode bits zero. The OS puts that address there once, at start-up:

```
csr_write(mtvec, (uintptr_t)trap_entry);    /* csrw */
```

`csr_write()` is a provided C helper around the `csrw` instruction. `trap_entry` is a **label** in the provided `trap.S`: the name of the first instruction of the trap path. Like every **symbol** listed with `nm -n` in Lecture 4, it gets its address from the linker when the linker places `.text`. `trap.h` declares it as `void trap_entry(void);` only so that C code can take its address; `(uintptr_t)trap_entry` is that address as a number for `csrw`. Nobody calls `trap_entry`, as the next section shows. The glossary distinguishes label and symbol.

The prefix `m` in all these names means machine mode, the most privileged RISC-V mode, in which our whole system runs. The OS decides once where all traps enter: at `trap_entry`.

5.3.7 On a return: `mret`

On a trap	On a return (`mret`, machine return)
record where: `mepc <- trap pc`	continue there: `pc <- mepc`
record why: `mcause <- reason`	nothing: `mcause` was only for the OS
interrupts off: `MPIE <- MIE`, `MIE <- 0`	previous state back: `MIE <- MPIE`

A return is not an undo of the trap. The record of where becomes the place to continue, the record of why simply stays, and the interrupt-enable bit gets back the value saved in `MPIE`. Architecturally, `mret` restores the previous interrupt-enable state; it does not by itself mean "interrupts on". It also sets `MPIE` to 1 and restores the privilege mode from `MPP`, in our system always machine mode. Whether interrupts are on after `mret` is therefore the OS's choice: our return path deliberately sets `MPIE` to 1 before every `mret`, so in our system every process runs with machine interrupts enabled. Lecture 6 depends on this.

If the OS changes `mepc` before `mret`, execution continues elsewhere. Compare it with `ret` in Lecture 3: `ret` jumps to `ra`; `mret` jumps to `mepc`. Both are ordinary instructions that use saved state. The OS can therefore return to the instruction after an `ecall`, or later to a different execution altogether. The `pc` is still never written like a register: `mret` sets it from `mepc`.

The CPU neither saves nor restores the process's registers: **the OS saves them after the trap and restores them before `mret`**. Only the process's registers need this. The OS has none to restore: each handler run starts fresh, and no OS execution is resumed later. In CP1 the saved OS context in `os_context` was exactly the red line; with traps it is gone.

5.4 Between trap and return: the OS's mechanisms

5.4.1 Why `ra`, `sp`, `s0`–`s11` no longer suffice

CP1 saved 14 values, and at a call boundary that was enough. Compiled code follows the RISC-V calling convention, a software agreement defined in the RISC-V ABI. The **caller**, the function that makes a call, must expect `t0`–`t6` and `a0`–`a7` to be destroyed and keeps

whatever it still needs in `s` registers or in its stack frame. The **callee**, the function being called, must hand back `sp` and `s0`–`s11` unchanged. Across a call, only `sp`, `s0`–`s11` and the continuation in `ra` matter; everything else the caller needs is in memory. Saving all 31 registers would also have been correct, just unnecessary.

The compiler guarantees this convention for the code it generates; it does not enforce it. Hand-written assembly such as CP1's `context_switch` has to keep it itself, and nobody checks. The hardware knows nothing of it, which is why the CPU's trap rule gives no such guarantee.

A trap is not a call, and the convention gives no guarantee there. The same entry path serves very different situations:

```
ecall in process_yield()   at a call boundary
illegal instruction        anywhere in the code
timer (Lecture 6)          between any two instructions
```

For example:

```
    add  t0, a0, a1     # t0 holds an intermediate value
    # <- timer interrupt here (Lecture 6)
    slli t0, t0, 3      # t0 is still needed
```

`t0` is caller-saved, but that only means a callee may destroy it. Here there is no callee, so nobody saved `t0`.

The same happens in compiled code, not only in hand-written assembly. Here is a loop that only counts, the kind of process Lecture 6 interrupts with the timer:

```
volatile unsigned long counter = 0;
while (counter < 1000000) {
    ++counter;
}
```

Our build (GCC 13.2, `-O2`) turns the loop into five instructions; the comments are ours:

```
800002b4:  ld    a5,8(sp)       load counter
800002b6:  addi  a5,a5,1        +1
800002b8:  sd    a5,8(sp)       store counter
800002ba:  ld    a5,8(sp)
800002bc:  bgeu  a4,a5,800002b4 loop while counter <= limit
```

The compiler loaded the limit 999999 into `a4` once, before the loop, and keeps it there for the whole loop. Between the first `ld` and the `addi`, the counter exists only in `a5`. Both are

argument registers, so caller-saved, and no call is in sight: nothing in the calling convention protects them. A trap between any two of these instructions must leave `a4` and `a5` exactly as they were, although the handler's C code is free to use both. The same holds for every other caller-saved register a compiler may use in this way: `ra`, `t0`–`t6` and `a0`–`a7`.

Even at our `ecall` the CP1 subset is not enough. Because `process_yield()` is an ordinary function, the call boundary does protect the caller's own live values: they are in `s` registers or in memory. But the system-call registers themselves are not among the 14 values: `a7` tells the handler which system call this is, and `a0` and `a1` carry the arguments and the result. The handler reads them from the frame, so the trap path must capture them. In addition, the assembly statement in `syscall.c` promises the compiler that only `a0` changes across the `ecall`; an inlined stub could keep further live values in caller-saved registers. This system-call contract is the OS's choice: the RISC-V ABI leaves system calls outside its scope.

One entry path serves all traps: it must save all 31 writable integer registers. It must also save them before any C code runs, because the handler is a fresh run of C code that immediately uses registers for its own work.

5.4.2 The trap frame replaces `struct context`

The complete saved integer context of an execution that trapped, as our teaching model requires it, is its **trap frame**:

```
struct trap_frame {
    uintptr_t regs[32];   /* regs[i] = xi; x0 unused */
    uintptr_t pc;         /* continuation address */
};
```

`regs[i]` holds register `xi`. `regs[0]` is unused because `x0` is hardwired to zero: it always reads as 0 and discards every write, so it carries no state of the process. The 32 slots keep the slot index equal to the register number, so `xi` lies at byte offset $8 \cdot i$. We build for `rv64imac`, without the floating-point extensions F and D, so "all registers" means the 31 writable integer registers.

The field `pc` is the saved continuation address for this execution, an ordinary value in memory; it is not a copy of the live `pc`. At trap entry it receives the trap `pc` from `mepc`. For an interrupt that already is the continuation address; for an `ecall` the handler converts it, as shown below. The layout is fixed: the provided assembly stores `xi` at byte offset $8 \cdot i$ and `pc` at offset 256.

The name is historical. In many systems every process has its own OS stack, and the trap entry pushes the saved registers onto it, where the block sits like the stack frame of a call (Lecture 4). Ours is not on any stack: `frame` is a field of `struct process`, next to `stack` but not inside it. In the rest of this chapter, "frame" alone means the trap frame; the call data of Lecture 4 is called "stack frame".

The trap frame replaces the CP1 context in the process object; the field order is otherwise unchanged:

```
struct process {
    process_entry_t entry;
    uint8_t stack[STACK_SIZE];
    struct trap_frame frame;      /* was: struct context context */
    enum process_state state;
    int exit_status;
};
```

	CP1 `struct context`	`struct trap_frame`
registers	`ra`, `sp`, `s0`–`s11`	31 integer registers
continuation	`ra`, restored by `ret`	`pc`, via `mepc` and `mret`
field	`struct context context;`	`struct trap_frame frame;`

The field is called `frame`, so that code reads `p.frame` and `frame->pc`; the type name says what kind of frame it is. The cooperative CP1 context was the special case of this frame that a function-call boundary allows.

5.4.3 The trap path replaces `context_switch()`: save

`context_switch()` saved and restored a context in one function. With traps, its two halves become two pieces of the provided `trap.S`: the **trap path**, which saves, and the **return path**, which restores. `context_switch()` saved 14 registers when the process called it; the trap path saves 31 registers plus the trap `pc` when the CPU jumps to it.

The trap path has to solve one problem first. At trap entry every register belongs to the process, yet every store such as `sd x5, 40(base)` needs an address in a register. Loading the frame address into any register would destroy a process value that is not yet saved, and `sp` belongs to the process as well. The solution is a CSR that already holds the address: `mscratch`, which has no meaning of its own and is written only by the OS. Before every `mret`, the first start included, the return path sets it to the frame it returns into.

```
trap_entry:                 # mscratch = &p.frame, set by OS
    csrrw t6, mscratch, t6  # swap: t6 = &p.frame
    sd    x1, 8(t6)         # x1 .. x30 into the frame
    sd    x2, 16(t6)
    sd    x3, 24(t6)
    sd    x4, 32(t6)
    sd    x5, 40(t6)
    sd    x6, 48(t6)
    sd    x7, 56(t6)
```

```
    sd    x8, 64(t6)
    sd    x9, 72(t6)
    sd    x10, 80(t6)
    sd    x11, 88(t6)
    sd    x12, 96(t6)
    sd    x13, 104(t6)
    sd    x14, 112(t6)
    sd    x15, 120(t6)
    sd    x16, 128(t6)
    sd    x17, 136(t6)
    sd    x18, 144(t6)
    sd    x19, 152(t6)
    sd    x20, 160(t6)
    sd    x21, 168(t6)
    sd    x22, 176(t6)
    sd    x23, 184(t6)
    sd    x24, 192(t6)
    sd    x25, 200(t6)
    sd    x26, 208(t6)
    sd    x27, 216(t6)
    sd    x28, 224(t6)
    sd    x29, 232(t6)
    sd    x30, 240(t6)
    csrr  t5, mscratch       # t6 of the process
    sd    t5, 248(t6)        #   as x31
    csrr  t5, mepc           # the trap pc
    sd    t5, 256(t6)        #   as frame->pc
    la    sp, _stack_top     # only now: the OS stack
    mv    a0, t6             # for trap_handler: &p.frame
    call  trap_handler       # runs on the OS stack
```

The first instruction exchanges `t6` and `mscratch`: `csrrw rd, csr, rs1` sets `rd` to the old CSR value and the CSR to the old value of `rs1`. Afterwards `t6` (`x31`) holds the frame address, and the process's `t6` waits in `mscratch`. Every store now has an address: `xi` goes to byte offset $8 \cdot \mathrm{i}$. The parked `t6` is fetched through `t5` (`x30`), which is free by then because it has already been stored, and saved as `x31`. Then the trap `pc` from `mepc` goes to offset 256.

The CPU leaves `sp` unchanged: a trap changes `pc`, `mepc`, `mcause` and `mstatus`, but no general-purpose register. Until `la sp`, `sp` still points into the process stack. Saving needs no stack, because the stores go through `t6` into `p.frame`. Only the C handler needs a stack for its stack frames, so only after everything is saved does `la sp, _stack_top` select the OS stack. The OS stack is not new: the linker reserves it, and `main()` has used it since Lecture 2. What is new is that a trap does not select it; the provided code does. The process's `sp` cannot be trusted in general, and OS data should not be mixed into process storage. Some architectures switch stacks in hardware, for example x86 when coming from user mode; RISC-V does not.

`trap_entry` and `trap_handler()` are different kinds of code. `trap_entry` is never called: the CPU jumps there via `mtvec`, between two arbitrary instructions of the process, so it can follow no calling convention, and it ends with `mret`. In its last two lines it calls `trap_handler()`, an ordinary C function that runs on the OS stack, receives the frame in `a0`, and returns the frame to restore in `a0`.

5.4.4 The return path replaces `context_switch()`: restore

The second half of `trap.S` follows directly after the call. Both listings are the complete provided code; only the file's header comment and assembler directives are left out, and the offset 256, which `trap.S` names `FRAME_PC`, is written as a number:

```
    call  trap_handler       # returns a0 = frame to restore
os_start:                    # os_run() enters here, too
    csrw  mscratch, a0       # a0: result of trap_handler
    ld    t5, 256(a0)
    csrw  mepc, t5           # where to continue
    li    t5, (3 << 11) | (1 << 7)   # MPP = M, MPIE = 1
    csrs  mstatus, t5
    ld    x1, 8(a0)          # x1 .. x31 from the frame, not a0
    ld    x2, 16(a0)
    ld    x3, 24(a0)
    ld    x4, 32(a0)
    ld    x5, 40(a0)
    ld    x6, 48(a0)
    ld    x7, 56(a0)
    ld    x8, 64(a0)
    ld    x9, 72(a0)
    ld    x11, 88(a0)
    ld    x12, 96(a0)
    ld    x13, 104(a0)
    ld    x14, 112(a0)
    ld    x15, 120(a0)
    ld    x16, 128(a0)
    ld    x17, 136(a0)
    ld    x18, 144(a0)
    ld    x19, 152(a0)
    ld    x20, 160(a0)
    ld    x21, 168(a0)
    ld    x22, 176(a0)
    ld    x23, 184(a0)
    ld    x24, 192(a0)
    ld    x25, 200(a0)
    ld    x26, 208(a0)
    ld    x27, 216(a0)
```

```
    ld    x28, 224(a0)
    ld    x29, 232(a0)
    ld    x30, 240(a0)
    ld    x31, 248(a0)
    ld    x10, 80(a0)        # a0 last: it held the frame
    mret                     # pc <- mepc, MIE <- MPIE
```

After `call trap_handler`, `a0` holds the returned frame, and the code simply continues at the label `os_start`, with no instruction in between. The label exists because `os_run()` calls the same code as `os_start()` for the first start of a process. The return path is one piece of code with two ways in: after every trap, and once from `os_run()` for the first start.

The first line makes `mscratch` point to the frame that is about to run, which prepares that process's next trap. The `li`/`csrs` pair sets `MPP` to machine mode (3, in bits 11–12) and `MPIE` to 1 (bit 7) in `mstatus`, so that `mret` stays in machine mode and, through `MIE <- MPIE`, enables machine interrupts for the process: our policy, not an effect of `mret` alone. A trap sets `MPP` itself, but the first `mret` from `os_start` needs it set explicitly. The loads skip `x10`: `a0` is restored last because it holds the frame address during all other loads.

5.4.5 `trap_handler()`: return the frame to restore

Between the two paths sits one C function:

```
struct trap_frame *trap_handler(struct trap_frame *trapped);
```

The trap path calls it with `a0` = `&p.frame`. It receives the saved state of the process that trapped and returns the state that runs next:

Returned frame	Meaning
the same one	resume
another one	schedule (Lecture 7)

This one signature carries the whole CP2 path. The provided assembly calls it; the OS's C code implements it. In Exercise 4 it comes from `libkernel.a`; for CP2 students write their own, extended by timer interrupts (Lecture 6) and a scheduler (Lecture 7). The parameter is called `trapped`, not `interrupted`: today every trap is synchronous. As in Lecture 3, a switch saves one context and restores another; the trap path only splits that into a hardware part, a provided assembly part, and a C decision in the middle.

5.5 Refactoring `main()` and `process_yield()` to use traps

5.5.1 `ecall`: a trap on purpose

`ecall` is an ordinary instruction whose only effect is a trap, with `mcause` 11. The provided `process_yield()`, compiled with GCC 14.2:

```
process_yield:
    li   a0, 0
    li   a1, 0
    li   a7, 1          # SYS_YIELD
    ecall               # -> mtvec, mcause = 11
    ret
```

Other compilers may order the three `li` differently. This is not hand-written assembly: `syscall.c` is C with one inline `ecall`.

A **system call** is a request to the OS made with `ecall`. We use the convention that `a7` selects the service and `a0`, `a1` carry arguments; the OS may return a result in `a0`. Keep the request apart from the function that makes it: `process_yield()` is the C function that makes the system call `yield`, just as the C library's `write()` in Linux is a function that makes the system call `write`. `process_yield()` and `process_exit()` now make our first two system calls, `yield` and `exit`. The application still calls them by name; only the code behind them changed.

5.5.2 Continue after the `ecall`

For an `ecall`, the trap `pc` is the address of the `ecall` itself. Restoring the frame unchanged would execute the `ecall` again, which traps again: the process would yield forever and never reach its next line, and the OS would print "yielded" endlessly. The handler therefore converts the saved trap `pc` into the continuation address:

```
trap_entry:   p.frame.pc = mepc   trap pc: the ecall itself
trap_handler: p.frame.pc += 4     now the continuation address
os_start:     mepc = p.frame.pc; mret
```

The left column names the code that does each step. The right side describes in C-like notation what that code does: only the middle line is real C, written in the handler as `trapped->pc += 4`, because `trapped` points to `p.frame`. The other two lines stand for the `csrr`/`sd` and `ld`/`csrw`/`mret` instructions in `trap.S`.

This is why the handler modifies the trap frame at all. An `ecall` is always 4 bytes long; it has no compressed 2-byte form, even with the C extension enabled in our build. Interrupts receive no such adjustment: there `mepc` already names the instruction that still needs to execute.

5.5.3 `yield` and `exit` as system calls

System call	`a7`	OS action
`yield`	1	report, `RUNNING -> READY`, resume
`exit`	0	record `a0` as status, `-> FINISHED`

With one process, `yield` marks the process `READY` and immediately selects it again. This looks pointless today; it becomes meaningful when there are several `READY` processes in Lecture 7. The exit status travels in `a0` and is read from the saved frame, not from the live register: by the time C code runs, the live `a0` holds the handler's own argument. The `os:` output lines now come from the trap handler:

```
os: process 1 yielded
os: process 1 exited 0
```

5.5.4 Inside `trap_handler()`

The whole handler as of today, condensed from the reference implementation:

```
uintptr_t cause = csr_read(mcause);   /* the reason */
if (cause == EXC_ECALL_FROM_M) {      /* 11: ecall */
    trapped->pc += 4;
    long a0 = trapped->regs[REG_A0];
    switch (trapped->regs[REG_A7]) {
    case SYS_YIELD: return reschedule();
    case SYS_EXIT:  return finish_current(a0);
    }
}
report_unexpected(cause, trapped->pc);
return finish_current(EXIT_STATUS_TRAP);
```

The OS does not know why it was entered until it reads `mcause`. This is the difference to a function call, where the callee knows why it was called because the caller chose it. All traps arrive at the same code, so the first line reads the reason. `EXC_ECALL_FROM_M` is 11: only an `ecall` produces this cause, and our processes execute `ecall` only to make a system call, so the branch is the system-call path. RISC-V distinguishes where an `ecall` came from: 8 from user mode, 9 from supervisor mode, 11 from machine mode. Our processes run in machine mode.

The `ecall` branch combines the two previous subsections: `+= 4` turns the trap `pc` into the continuation address, and `a7` selects the system call. Every branch ends by returning a frame, as the contract requires. `reschedule()` and `finish_current()` are the reference's

helpers: they update the state of the current process (`READY`, or `FINISHED` with a status), choose the next process and return its frame. With one process, `yield` returns the same frame; after `exit`, only the idle process is left, a `wfi` loop that the OS starts when nothing else is `READY`. An unknown system-call number leaves the `switch` and also ends the process with an error status.

`report_unexpected()` is a name used only here for brevity; in the reference implementation the report is inline code. The reference also has an interrupt branch before the `ecall` test, omitted here because no interrupt is enabled in the Exercise 4 stage. Lecture 6 adds the timer branch. Today's handler is the core that students extend for CP2, not the CP2 handler.

5.5.5 Unexpected traps

A faulty instruction in the process enters through the same `mtvec`. Inserted into `worker()`, the following line places the 32-bit word 0, an encoding defined as illegal on RISC-V, into the code:

```
__asm__ volatile (".word 0");   /* illegal instruction */
```

The CPU traps with `mcause` = 2 and `mepc` = the address of the inserted word. `trap_handler()` does not find `EXC_ECALL_FROM_M`, so it reaches its last two lines: it reports the cause and the saved `pc`, marks the process `FINISHED` with an error status, and returns the next frame. With one process, that is the idle loop. The output, from a GCC 14.2 build, is:

```
trap: cause 2 at 0x80000136 in process 1
```

The first code line runs in the process; the output line is printed by the OS. The address is build-specific and can be located with `objdump -d`. One faulty process does not take down the OS. Without this case, the handler would have to guess, and a fault would look like a system call or vanish.

5.5.6 First entry: prepare a frame

A process that has never run has no saved state yet. As in CP1, the OS prepares one:

CP1 initial context	Trap-frame equivalent
`p.context.ra = process_start`	`p.frame.pc = process_start`
`p.context.sp = stack top`	`p.frame.regs[2] = stack top`
`p.context.s[i] = 0`	all other `p.frame.regs[i] = 0`

The new process looks as if it had trapped into the OS just before its first instruction. Every assignment writes a field in memory, not a register: `p.frame.pc` is the saved continuation

address, and the live `pc` receives this value only later, when the return path copies it into `mepc` and executes `mret`. `regs[2]` is `sp` (`x2`); the stack top must be 16-byte aligned, as in CP1. `process_start()` is still the internal bootstrap: it calls the entry function and, if that returns, `process_exit(0)`. The first restore of this frame enters `process_start()`; this is the first-start return of the CPU-box lifetime.

5.5.7 Three entries

In CP1, `main()` simulated the OS. It called `process_run()`, got control back when the process yielded, and printed `os: process yielded` itself. With traps, a yield lands in `trap_handler()`, not in `main()`. The model "the OS is a program that calls processes" no longer fits. Instead, the OS is code that runs when an event occurs. `main()` therefore only starts the system:

```
void main(void)                 /* the OS's start-up */
{
    static struct process p;

    process_create(&p, worker);
    os_run();                   /* never returns */
}
```

`os_run()` installs `mtvec`, selects the first process and calls the provided `os_start()`, which restores that process's frame and executes `mret`: the first return. `main()` never gets control back, neither by a return nor by a resume. The system now has three entries:

```
_start      CPU's entry      once, at reset; calls main()
main()      OS start-up      once; never returns
trap_entry  OS entry         every trap, via mtvec
```

`_start` (in `start.S`) sets up the OS stack and calls `main()`. `_start` is the CPU's entry, `main()` is the OS's start-up, and the trap vector is the OS's entry for everything that happens afterwards. The trap vector is the address in `mtvec`, and that address is `trap_entry`. `main()` is where the OS begins, not where it lives.

Real operating systems follow the same pattern at this level: a one-time start-up, followed by event-driven entry into the kernel. In Linux, the start-up function `start_kernel()` initialises the system, including the trap vector. Through `rest_init()` it then creates two kernel threads, `kernel_init` and `kthreadd`, and the boot task itself becomes the idle task. `kernel_init` finally loads the first user program, `init`, which becomes PID 1; it enters user mode through a return instruction (`sret`, the supervisor-mode counterpart of `mret`). From user space the kernel is entered only through traps, and further processes are created by `init` and its descendants through the system calls `fork` and `exec`. The mechanisms differ in detail; the shape is the same. Our `main()` still creates the processes itself, because we have no system call for that yet.

5.5.8 Why `p` must become static

Exercise 2 declared `struct process p;` inside `main()`, an object with automatic storage duration. Its storage, including `p.stack` and the saved state, was part of `main()`'s stack frame at the top of the OS stack:

```
OS stack
+--------------------------------------+  _stack_top <- sp
| main()'s stack frame: p, p.stack ... |
+--------------------------------------+
```

In CP1 this was safe: `main()` stayed active, and the OS ran below its stack frame. Now every trap executes `la sp, _stack_top`, and `trap_handler()` builds its stack frames downward from `_stack_top`, into the region where `p` lies. `main()` is never resumed, so nothing treats its stack frame as live any more. The handler's own call data overwrites the top of `p`, including parts of `p.frame` and `p.state`, at the first trap.

Whether that becomes visible depends on the build. With GCC 14.2 the output may even stay correct, because the overwritten saved registers happen to be unused by this small program; with clang the output stops after `os: process 1 yielded`. A run that looks correct is not a correctness argument: the saved state is no longer reliable, and Exercise 4 lets you find the overlap in the disassembly.

Declaring `p` as `static` gives it static storage duration: the object exists once, for the whole run, in `.bss`, outside every stack. A global variable outside `main()` has the same storage duration; `static` inside `main()` only keeps the name local, while the OS reaches the object through `&p` anyway. The rule behind this: anything that must outlive a single handler run must not live on the OS stack. The glossary defines both storage durations.

5.6 The whole path

5.6.1 The lifetime in detail

The CPU-box lifetime from the beginning of the chapter, now with the code names:

```
              OS             CPU               Process
                           +--------+
main()        | os_run()   |        |
os_start      +----------->| return |---------->| (start)
                           |        |             | ecall
trap_entry    |<-----------|  trap  |<------------+
              | save frame |        |
trap_handler  | pc += 4    |        |
os_start      +----------->| return |---------->| (resume)
```

`exit` takes the same path; its handler returns the idle process's frame. "start-up" became `main()` and `os_run()`, each "handler" bar became three pieces of code, and "prepare" and "decide" are now concrete. The CPU box is unchanged: the CPU traps and returns; the OS saves, decides, and restores.

The same steps as a list of what runs where:

```
permute() calls process_yield()   on the process stack
  ecall                           CPU: mepc, mcause, pc <- mtvec
trap_entry: save 31 regs + pc     into p.frame
  la sp, _stack_top               now on the OS stack
trap_handler: cause 11, a7 = 1    pc += 4; READY, then RUNNING
  return &p.frame
os_start: restore p.frame, mret   pc = after the ecall
process_yield() returns           same permute() call
```

After `mret`, `sp` selects the process stack again because it was restored from the frame. `process_yield()` returns into the same `permute()` invocation, which continues with the same array.

5.6.2 Our trap model: who does what

Step	Who	What
trap	CPU	`mepc`, `mcause`; interrupts off; `pc <- mtvec`
save	`trap_entry`	registers and trap `pc` into the frame; OS stack
decide	`trap_handler()`	read `mcause`, return a frame
restore	`os_start`	registers and `mepc` from the frame
return	CPU, `mret`	`pc <- mepc`; `MIE <- MPIE`

The CPU's two rules frame the OS's three pieces of code. `trap_entry` and `os_start` are provided assembly; for CP2, students write their own `trap_handler()`, with timer and scheduler (following Lectures 6 and 7).

This is a deliberately restricted teaching model of trap-based OS entry, not a complete model of the RISC-V privileged architecture. Inside the model:

- execution remains entirely in machine mode, with one trap entry point through `mtvec`;
- the traps covered are `ecall`, timer interrupts (Lecture 6), and unexpected traps such as illegal instructions;
- the CSRs our trap mechanism uses are `mtvec`, `mepc`, `mcause` and `mstatus`, which the hardware trap rule reads or writes, and `mscratch`, which the hardware never touches on a trap and our OS uses by convention; Lecture 6 adds the timer, a device of the platform, not of the CPU;

- the trap transition itself is hardware; saving the process's state, switching to the OS stack, calling the C handler and restoring the selected frame are software, in the provided assembly, which ends with `mret`;
- the trap frame holds the 31 integer registers and the continuation address;
- scheduling is choosing which trap frame is restored.

Outside the model: privilege modes and protection, delegation of traps between privilege levels, nested traps, virtual memory and page faults, further interrupt sources, floating-point state, and production-grade interrupt handling. Because everything runs in machine mode, nothing prevents a process from changing `mtvec`, the timer or OS memory; we accept this to keep the mechanism small and visible. The essential mechanism is this: an event transfers control into the OS, the state of the trapped execution becomes explicit data, and the OS decides which saved state continues.

5.7 Key ideas

- A **trap** is a transfer of control into the OS that the CPU performs itself; exceptions come from the current instruction, interrupts from independent events, and both enter at `mtvec`.
- On a trap the CPU records where in `mepc` (the trap `pc`) and why in `mcause`, turns interrupts off and jumps to `mtvec`; `mret` continues at `mepc` and restores the previous interrupt state, which our return path sets to "enabled". Only `mret` sets the `pc` from `mepc`.
- The **trap frame** replaces `struct context`: 31 integer registers plus the continuation address, because a trap is not a call and the calling convention gives no guarantee.
- The **trap path** and the **return path** in `trap.S` replace `context_switch()`; the trap path, not the CPU, switches to the OS stack, and only after everything is saved.
- A **system call** is a request made with `ecall`; the handler adds 4 to the saved `pc` to obtain the continuation address.
- `trap_handler()` reads `mcause` and returns the frame that runs next: the same one resumes, another one schedules.
- `main()` is only the OS's start-up; afterwards the OS is entered only at `trap_entry`, and nothing of the OS is suspended while a process runs.
- Our system runs entirely in machine mode; the trap mechanism is a deliberately restricted teaching model.

Related Exercise

Exercise 4 — One Trap Path ports the Exercise 2 application to the trap path and traces one system call with real addresses.

References

- RISC-V International, *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture*: machine-mode CSRs, trap entry, `mret`.
- RISC-V International, RISC-V ABIs Specification: caller- and callee-saved registers.
- Arpaci-Dusseau and Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, chapter *Mechanism: Limited Direct Execution*.

6 Timer and Preemption

Lecture 5 gave the OS a way in: the trap. Every trap in Lecture 5 was caused by the process, either on purpose with an `ecall` or by mistake with a faulty instruction. A process that never yields still keeps the CPU. This chapter answers the second half of the problem: **how does the OS get control back without the process's cooperation?** The answer is an event that no process can prevent, the passing of time, delivered as a timer interrupt through the same trap path.

We keep the Exercise 4 port. The only process is now a `spinner()` that counts, prints and exits, and never yields. There is still one process; the handler still returns the same frame. Everything of Lecture 5 stays, and only the source of the event changes.

The HTML version of this script contains interactive models for this chapter.

6.1 Where we stand

6.1.1 The lifetime through the CPU, recalled

Lecture 5 drew the lifetime of a process with the CPU as a box between the OS and the process:



The OS prepares a frame and returns into it; the process runs; the CPU traps at the `ecall`; the handler adds 4 to the saved `pc` and returns the same frame. Nothing of the OS is suspended while the process runs. Both kinds of trap we have seen are synchronous: they happen at the same instruction in every run, and the process is their cause.

6.1.2 Why a never-yielding loop is still stuck

```
for (;;) { /* keep computing, never call */ }
```

Lecture 4 ended with this loop and Lecture 5 began with it. The trap path now exists, but nobody enters it: the loop executes no `ecall` and no faulty instruction, so the CPU never traps. Lecture 5 gave us the way in, not a reason to take it.

What the OS needs is an event that happens whatever the process does. A key press, a network packet or another device could interrupt the process, but none of them is guaranteed to happen. Time passes in any case. That is the event this course uses.

6.1.3 Two halves of the answer

Lecture 5 split the answer to the never-yielding process into two halves:

```
Lecture 5: the process asks       ecall   -> CPU -> OS
Lecture 6: an event interrupts    timer   -> CPU -> OS
```

Lecture 5 promised that the second half reuses the identical entry path. This chapter keeps that promise. Three things are new: a device that keeps time and raises an interrupt request at a set time, the conditions under which the CPU accepts that request, and the two places where the trap path behaves differently for an interrupt. Today's trap is asynchronous and non-cooperative at once: the process neither causes it nor intends it.

Keep the distinction from Lecture 4 in view. **Interrupt handling** is how the OS gets control; the **scheduling decision** is what it does with it. With one process the decision is trivial. Whether it must stay trivial is the question this chapter leaves for Lecture 7.

6.2 Time as an event: the platform timer

6.2.1 A device you have used since Exercise 1: the UART

Every program of this course has printed through the UART, the serial device of our machine. Lecture 2 left its code aside; the timer works the same way, so we look at it now. The starter `main.c` of Exercise 1 contained:

```
#define UART0_BASE 0x10000000UL
#define UART_THR    0
#define UART_LSR    5
#define UART_LSR_THRE 0x20
```

```
static volatile uint8_t *const uart0 = (volatile uint8_t *)UART0_BASE;

static void uart_putc(char c)
{
    while ((uart0[UART_LSR] & UART_LSR_THRE) == 0) {
    }

    uart0[UART_THR] = (uint8_t)c;
}
```

`uart0` is a pointer to the first of the UART's byte-wide registers, at address `0x10000000`. The store `uart0[UART_THR] = (uint8_t)c` writes one byte to `0x10000000`. No memory cell receives it: the UART takes the byte and sends it to the terminal. The load `uart0[UART_LSR]` reads one byte from `0x10000005`, the UART's line status register; its bit `0x20` says that the UART is ready to accept the next character. The loop asks the device again and again until it is ready. This is called **polling**.

So an address does not have to lead to RAM. A device register that has an address, and is read and written with ordinary loads and stores, is called **memory-mapped**. Since Exercise 4, `uart_putc()` lives in the shared `uart.c`, with the same code.

6.2.2 The timer is a platform device

Lecture 5 separated the CPU from the **platform**, the machine around it. The CPU provides CSRs, instructions and the trap rule; the platform provides memory and devices, and decides which address leads where. For our command line (`-machine virt -m 128M`), QEMU's `virt` machine routes loads and stores as follows:

```
address range            what answers   role
0x02000000 - 0x0200ffff  CLINT          keeps time (this chapter)
0x10000000 - 0x100000ff  UART           serial output
0x80000000 - 0x87ffffff  RAM            program and data
```

RAM starts at `0x80000000`, the `ORIGIN` of the linker script from Lecture 2, and `-m 128M` makes it 128 MiB long (`0x8000000` bytes). The UART's `0x10000000` is therefore not inside RAM: it lies far below RAM's start, and a load or store there reaches the UART. The other ranges of the address space hold further `virt` devices that our system does not use, or nothing at all. The CSRs of Lecture 5 do not appear in the map: they have no address.

None of these devices is created by our code. `-machine virt` selects a board that QEMU emulates as a whole, with its devices at addresses fixed by QEMU's model of that board; QEMU also describes them in the device tree it generates. A real chip has its devices built in the same way, at addresses its manual documents. Our code only uses the addresses: `0x10000000` in `uart.c`, and the CLINT's addresses in `riscv.h`.

Lecture 4 named three parts of our target: CPU, RAM and UART. The timer is the fourth part we use. The **CLINT** (core-local interruptor) is the device of QEMU's `virt` machine that keeps time and requests timer interrupts from the CPU. The CPU has no timer of its own that our OS programs; it reacts to the CLINT's request as it would to any interrupt source. Newer RISC-V documents describe the same registers under the name ACLINT; QEMU and our code use CLINT. The addresses are those of QEMU's `virt` machine. Real boards place their devices elsewhere, which is why the platform, not the ISA, defines them.

The roles are split. The device decides when to request an interrupt; the CPU decides when to take it.

6.2.3 `mtime` and `mtimecmp`

The CLINT has two registers that matter for us:

Register	Address	Holds
`mtime`	`0x0200bff8`	the current time: counts up at 10 MHz
`mtimecmp`	`0x02004000`	the time of the next timer interrupt

`mtime` counts up from 0 at machine start, on QEMU `virt` at 10 MHz: 10,000,000 counts per second, one count every 100 ns, 10,000 counts per millisecond, whatever the CPU executes. The device counts, not the OS: no instruction of our system increments `mtime`. It never stops; machine mode could write it, but our OS never does. `mtimecmp` holds a point in time, not a duration. The CLINT compares the two continuously: **when `mtime >= mtimecmp`, the CLINT requests a timer interrupt.** Both registers are 64 bits wide. As an unsigned 64-bit counter at 10 MHz, `mtime` would reach its largest value $2^{64} - 1$ only after about 58,000 years and then wrap to 0, so overflow plays no role.

Each hart has its own `mtimecmp`; all harts share one `mtime`. A **hart** (hardware thread) is the RISC-V word for one CPU that executes instructions. Our machine has one hart, so we use the `mtimecmp` of hart 0.

Both registers are memory-mapped, like the UART's: an ordinary load or store to their address reads or writes the device instead of memory. The CSRs of Lecture 5 are different: they have no address and are reached only through the `csr…` instructions. Both are hardware state, reached in two different ways.

6.2.4 Reading and writing a device register in C

The provided `riscv.h` turns the two addresses into pointers:

```
#define CLINT_MTIMECMP0 ((volatile uint64_t *)0x02004000UL)
#define CLINT_MTIME     ((volatile uint64_t *)0x0200bff8UL)
#define TIMER_HZ        10000000UL

static inline void timer_arm(uint64_t delta)
{
    *CLINT_MTIMECMP0 = *CLINT_MTIME + delta;
}
```

Read the declarations from the inside out:

- `0x02004000UL` is only a number, an `unsigned long` constant (suffix `UL`).
- The cast `(volatile uint64_t *)` turns that number into a pointer: the address of a 64-bit unsigned value. Nothing is read yet.
- `#define` gives the pointer expression a name. The preprocessor pastes it into the code wherever the name appears; it is not a variable and occupies no memory.
- `*CLINT_MTIME` dereferences the pointer: a 64-bit load from `0x0200bff8`, which the CLINT answers with the current time.
- `*CLINT_MTIMECMP0 = …` on the left of an assignment is a 64-bit store to `0x02004000`.
- `static inline` in a header gives every `.c` file that includes `riscv.h` its own copy of `timer_arm()`, which the compiler may place directly at the call. There is no link conflict, and no call needs to happen.

The one line in `timer_arm()` is therefore a load from `mtime`, an addition, and a store to `mtimecmp`: it requests the next timer interrupt `delta` counts from now. It is the same pattern as `uart0[UART_THR] = c` in the UART code, with a 64-bit value instead of a byte. Our build (GCC 13.2, `-O2`) translates `timer_arm(100000)` into exactly these three steps; `lui` builds the addresses like any other constant:

```
lui  a4,0x200c       # a4 = 0x0200c000
ld   a5,-8(a4)       # load mtime from 0x0200bff8
lui  a3,0x18
addi a3,a3,1696      # a3 = 0x186a0 = 100000
add  a5,a5,a3        # mtime + delta
lui  a4,0x2004       # a4 = 0x02004000
sd   a5,0(a4)        # store to mtimecmp
```

Two details of the declaration matter. `uint64_t`, from `stdint.h`, says the width exactly: the register is 64 bits by definition, so the code says so, although `unsigned long` is also 64 bits under our `lp64` ABI. `volatile` tells the compiler that every access has an effect outside the program. Without it, the compiler may assume that memory changes only when the program writes it; it could load `mtime` once and reuse the value, or drop a store to `mtimecmp` that is never read back. Both are correct optimisations for RAM, of the kind Lecture 4 met with `-O2`, which keeps values in registers, and wrong for a device. With `volatile`, every

access really happens, in the order written. It does not make an access atomic and has nothing to do with disabling interrupts.

The parameter of `timer_arm()` counts `mtime` units. It is called `delta`, and the word "tick" is kept for something else later in this chapter.

6.2.5 The time slice

A **time slice** is how long the OS lets a process run until the next timer interrupt. Other books call it a *quantum*. Our OS uses 10 ms. `mtime` counts at `TIMER_HZ` = 10,000,000 counts per second, so 10 ms are 10,000,000 / 100 = 100,000 counts. The reference names this value in its `kernel.c`:

```
/* Time slice: 10 ms at the 10 MHz CLINT timebase, in mtime counts. */
#define TIME_SLICE (TIMER_HZ / 100)
```

`timer_arm(TIME_SLICE)` therefore writes `mtime + 100000` into `mtimecmp`. Because `mtimecmp` is a point in time, the OS adds the time slice to the current value of `mtime`. 10 ms is a common order of magnitude for real systems as well; nothing in our mechanism depends on the exact value.

6.2.6 Pending: a request, not yet a trap

Take `mtimecmp` = 100000 as an example:

```
mtime       99998   99999   100000   100001   100002 ...
pending       no      no      yes      yes      yes  ...
```

A timer interrupt is **pending** while `mtime >= mtimecmp`: the device has requested it, and the request has not been withdrawn. Pending is a level, not a single pulse. `mtime` keeps counting, so the condition stays true until software writes a larger `mtimecmp`. Nothing clears it by itself, and taking the interrupt does not clear it either. This has a consequence for the handler, which the section on the timer branch draws.

Three words stay apart from here on. An interrupt is **pending** when the device requests it, **enabled** when the CPU's enable bits allow it, and **taken** when the CPU applies the trap rule. Whether a pending interrupt is taken is decided by the CPU, as the next section shows.

6.3 Letting the interrupt in

6.3.1 Three conditions for taking a timer interrupt

Lecture 5 said that a timer interrupt is taken only if it is pending and two enable bits are set, and that Lecture 6 shows where "pending" is recorded. All three conditions live in CSRs:

Condition	Recorded in	Set by
pending	`mip.MTIP` (bit 7)	the CLINT, while `mtime >= mtimecmp`
timer switch on	`mie.MTIE` (bit 7)	the OS: `csr_set(mie, MIE_MTIE)`
global switch on	`mstatus.MIE` (bit 3)	the CPU's rules: trap and `mret`

`mip` (machine interrupt pending) and `mie` (machine interrupt enable) are CSRs with one bit per interrupt source. The bit for the machine timer is bit 7 in both: `MTIP` (machine timer interrupt pending) in `mip`, `MTIE` (machine timer interrupt enable) in `mie`.

- **Pending** is recorded in `mip.MTIP`. The OS does not write it. The CLINT's request sets it, and it clears when `mtimecmp` is moved beyond `mtime`. We need `mip` only as the place where pending is recorded.
- **Timer switch**: `mie.MTIE`, "enabled for the timer". The OS sets it once, during start-up; `MIE_MTIE` is the mask `1 << 7` from `riscv.h`, and `csr_set()` is the `csrs` instruction of Lecture 5.
- **Global switch**: `mstatus.MIE` from Lecture 5, "enabled globally". Our OS does not write it directly. The CPU's trap rule clears it, and `mret` restores it from `MPIE`, which the return path sets before every `mret`.

When all three hold, the CPU takes the interrupt before the next instruction: it applies the trap rule of Lecture 5 instead of executing that instruction. As a reminder, the rule has three steps:

```
record     mepc <- pc, mcause <- reason
off        mstatus: MPIE <- MIE, MIE <- 0
jump       pc <- mtvec
```

For an interrupt, the `pc` recorded in `mepc` is that of the instruction that has not executed yet. These rules are those for machine mode, the only mode in our system. The specification adds rules for other privilege modes and for several pending interrupts at once; neither occurs here.

6.3.2 Two switches in series: `mie.MTIE` and `mstatus.MIE`

The two enable bits are not equals. One belongs to the timer; the other stands above every interrupt source:

```
CLINT: mtime >= mtimecmp
   |   mip.MTIP = 1           pending
   v
[ mie.MTIE ]      timer switch    the OS sets it once
   |              (each source has its own bit in mie)
   v
[ mstatus.MIE ]   global switch   trap: 0, mret: 1
   |              (one bit for all sources)
   v
CPU applies the trap rule
```

The device requests, the request is recorded as pending, and it reaches the CPU only through both switches. Both must be on (1); neither is the pending bit. The **timer switch** `mie.MTIE` belongs to one source and stays as the OS set it. The **global switch** `mstatus.MIE` applies to all sources; the CPU itself turns it off on a trap and on again at `mret`. Logically the three conditions are simply combined with "and"; the hierarchy lies in the scope of the bits, one per source below one for all.

The names invite confusion. Lecture 5 introduced `mstatus.MIE`, bit 3 of `mstatus`, as "interrupts allowed (1) or not (0)", and announced a second, similarly named CSR. The lower-case `mie` is that whole CSR, with one enable bit per interrupt source; the timer's bit is `MTIE`, bit 7. The names come from the specification and cannot be changed, so we always write `mie.MTIE` or `mstatus.MIE`, never `MIE` alone.

`mip` and `mie` use the same bit position for the same source on purpose, so `MTIP` and `MTIE` are both bit 7. That `MPIE` in `mstatus` is also bit 7 is a coincidence.

6.3.3 Who sets which bit in our system

The OS does its part once, at start-up. Condensed from the reference `os_run()`, in the version of the Exercise 5 library:

```
void os_run(void)             /* start-up, once */
{
    csr_write(mtvec, (uintptr_t)trap_entry);
    timer_arm(TIME_SLICE);    /* first request in 10 ms */
    csr_set(mie, MIE_MTIE);   /* csrs: timer switch on */
    ...
    os_start(...);            /* mret: MIE <- MPIE = 1 */
}
```

`csr_write()`, `csr_read()` and `csr_set()` come from the provided `riscv.h`, as since Exercise 4: macros around the `csrw`, `csrr` and `csrs` instructions. `csr_set()` sets the given bits in a CSR and leaves the others unchanged. The dots stand for preparing the idle process's

frame, printing `os: starting 1 process`, and selecting the first process, as in Lecture 5. In the Exercise 4 library the two middle lines are missing.

While `os_run()` runs, the global switch `mstatus.MIE` is still 0 from reset. Nothing can be taken there, so the order of arming and enabling does not matter; what matters is that both happen before the first `mret`. That `mret` copies `MPIE`, which `os_start` set to 1, into `mstatus.MIE`. The first timer interrupt therefore arrives in the process, one time slice after `os_run()` armed the timer. The OS arms the timer and turns on the timer switch; `mret` turns on the global switch.

6.3.4 The global switch over time: the payoff of "interrupts off"

Lecture 5 taught the transition of the global switch `mstatus.MIE` on a trap and on `mret` exactly, and nothing in its output depended on it. The trap rule turns it off, `mret` turns it on again. Here is the payoff:

```
process runs   mstatus.MIE = 1         timer may interrupt
trap           MPIE <- MIE, MIE <- 0
OS runs        mstatus.MIE = 0         a request waits
mret           MIE <- MPIE, so 1       timer may interrupt
```

A timer interrupt can arrive between any two instructions. If it could arrive in `trap_entry`, a second trap would overwrite `mepc` and `mcause` before the first ones are saved. Because the trap rule clears `mstatus.MIE`, nothing interrupts the OS while it saves the process's state. A timer interrupt that becomes pending while the OS runs stays pending in `mip.MTIP` and is taken right after `mret`, before the process executes its next instruction. It is delayed, not lost.

In our system the OS never runs with interrupts enabled. This is a property of our model, in which nested traps are outside the scope, not a rule for every OS.

6.3.5 What held in Exercise 4

The Exercise 4 library never wrote `mtimecmp` or `mie`. While the process ran, the three conditions stood as follows:

Condition	In Exercise 4, while the process ran
`mstatus.MIE`	1: `os_start` sets `MPIE` before every `mret`
`mie.MTIE`	0: 0 at reset on QEMU, and never set
pending: `mip.MTIP`	1: set from reset on QEMU

The first row was already at work in Lecture 5. The second row alone kept the timer out: the timer switch was off. The third row is the surprise. The RISC-V specification fixes only

`mstatus.MIE` = 0 at reset and leaves `mie` and the reset value of `mtimecmp` open; QEMU starts all three at 0. Then `mtime >= mtimecmp` holds from reset, and `mip.MTIP` stays set for the whole run. A small probe program in the reference, built for our platform, prints `mtimecmp = 0x0` and `mip = 0x80` at reset, `mip = 0x0` after arming, and `mip = 0x80` again after expiry.

So in Exercise 4, exactly one bit was missing. The natural answer "not pending, because nobody armed the timer" is wrong on QEMU, and it would be unsafe as a design rule: an OS must not rely on an unspecified reset value. Had the Exercise 4 library set `mie.MTIE` without arming the timer, the interrupt would have been taken right after the first `mret`, and again after every return, because nothing would ever have moved `mtimecmp` into the future. Arming first and turning on the timer switch second, both before the first `mret`, makes the first interrupt arrive exactly one time slice after start-up.

6.4 The same trap path, now asynchronous

6.4.1 The CPU takes the interrupt

When the three conditions hold, the CPU applies the trap rule of Lecture 5. Side by side with an `ecall`:

Step of the trap rule	After `ecall`	After a timer interrupt
record where: `mepc`	the `ecall` itself	the next instruction, not executed
record why: `mcause`	11	interrupt bit set, code 7
interrupts off	`MPIE <- MIE`, `MIE <- 0`	the same
jump: `pc <- mtvec`	`trap_entry`	the same `trap_entry`

Only the first two rows differ, and only in the values recorded. The interrupt is taken between two instructions: the CPU has finished one instruction and, instead of executing the next one, applies the trap rule. That next instruction becomes the trap `pc`. `trap_entry` then saves all registers and the trap `pc` into the frame, exactly as after an `ecall`; it does not know which kind of trap it is saving, and it does not need to. Nothing is new in the CPU: the same rule, a different cause.

6.4.2 `mcause`: interrupt bit first

Lecture 5 introduced the layout of `mcause`: bit 63 is the interrupt bit, bits 62..0 hold the code. For the machine timer:

```
mcause = 0x8000000000000007
         bit 63 = 1: interrupt
         code   = 7: machine timer
```

mcause	Interrupt bit	Code	Meaning
0xb	0	11	ecall from M-mode
0x7	0	7	store/AMO access fault
0x8000000000000007	1	7	machine timer interrupt

The code 7 alone is ambiguous. Exception 7 is a store or AMO (atomic memory operation) access fault, for example a store to an address where nothing is mapped. A handler that compared only the code would treat such a fault as a timer interrupt and resume the faulting instruction forever. The handler therefore tests the interrupt bit first. `riscv.h` provides the names: `MCAUSE_INTERRUPT` is `1UL << 63`, `MCAUSE_CODE(c)` clears that bit, and `IRQ_MACHINE_TIMER` is 7.

6.4.3 The trap `pc` is the continuation address

Today's test process is the **spinner**, the process of Exercise 5. It is the never-yielding loop of Lecture 4, made concrete: it counts to a million and never calls `process_yield()`.

```
static void spinner(void)
{
    volatile unsigned long counter = 0;

    uart_puts("spinner: start\n");
    while (counter < 1000000) {
        ++counter;
    }
    /* prints "spinner: done, counter = ...", then exits */
}
```

The spinner has nothing to do with the timer mechanism itself. It is only a process that keeps the CPU busy, and its final counter lets us check afterwards that no increment was lost. Its loop, as compiled in our build (GCC 13.2), with our comments:

```
800002b4:  ld    a5,8(sp)       load counter
800002b6:  addi  a5,a5,1        +1
800002b8:  sd    a5,8(sp)       store counter
800002ba:  ld    a5,8(sp)
800002bc:  bgeu  a4,a5,800002b4 loop while counter <= limit
```

`counter` is a `volatile unsigned long` on the spinner's stack at `8(sp)`, so every iteration loads it, increments it and stores it back. `a4` holds the limit 999999, loaded once before the loop. The addresses are build-specific; a GCC 14.2 build, or your own `main.c`, places the loop elsewhere.

Suppose a timer interrupt is reported at `0x800002b6`. The `addi` at that address **has not executed yet**: the CPU took the interrupt instead of executing it. Resuming at the trap `pc` executes it for the first time. After an interrupt, `mepc` already says where to continue: the trap `pc` is the continuation address, and the handler changes nothing.

Lecture 5 had the opposite case. After an `ecall`, `mepc` holds the `ecall`, which has executed, and the handler adds 4. Adding 4 after an interrupt would skip work that was never done. Our target has the C extension, so instructions are 2 or 4 bytes long. From `0x800002b6`, `+ 4` lands on `0x800002ba`: `addi` and `sd` are skipped, and one increment is lost silently. From `0x800002bc`, `+ 4` skips the loop's branch, and the loop ends early. From a 2-byte instruction followed by a 4-byte one, `+ 4` lands in the middle of an instruction, and the CPU decodes from there.

The registers of the interrupted loop survive the handler for the reason Lecture 5 gave: between the `ld` and the `addi`, the counter exists only in `a5` and the limit only in `a4`, two caller-saved registers with no call in sight, and `trap_entry` saves all 31 registers before any C code runs. The Lecture 5 chapter uses this loop as its compiled example. The final line `spinner: done, counter = 1000000`, identical with and without timer interrupts, is the evidence in our run.

6.4.4 The timer branch of `trap_handler()`

The handler of Lecture 5 ("Inside `trap_handler()`") gets one new branch at the top; everything below it is unchanged:

```
uintptr_t cause = csr_read(mcause);   /* the reason */
if (cause & MCAUSE_INTERRUPT) {                    /* new */
    if (MCAUSE_CODE(cause) == IRQ_MACHINE_TIMER) {
        report_tick(trapped->pc);   /* Exercise 5 only */
        timer_arm(TIME_SLICE);      /* re-arm */
        return reschedule();        /* same frame, no +4 */
    }
} else if (cause == EXC_ECALL_FROM_M) {  /* 11: ecall */
    trapped->pc += 4;
    long a0 = trapped->regs[REG_A0];
    switch (trapped->regs[REG_A7]) {
    case SYS_YIELD: return reschedule();
    case SYS_EXIT:  return finish_current(a0);
    }
}
report_unexpected(cause, trapped->pc);
return finish_current(EXIT_STATUS_TRAP);
```

`report_tick()` and `report_unexpected()` are names used only here for brevity; in the reference both reports are inline code. `report_tick()` prints the line `os: tick N at <pc> in process 1` and is compiled only into the Exercise 5 library; the reference also counts the

interrupt there. In the reference, the `ecall` branch is the function `handle_syscall()`: it adds 4 and decides by `a7` as above, with additional report lines. An interrupt with another code falls through to the unexpected-trap report, as every unknown cause did in Lecture 5.

The timer branch has three jobs:

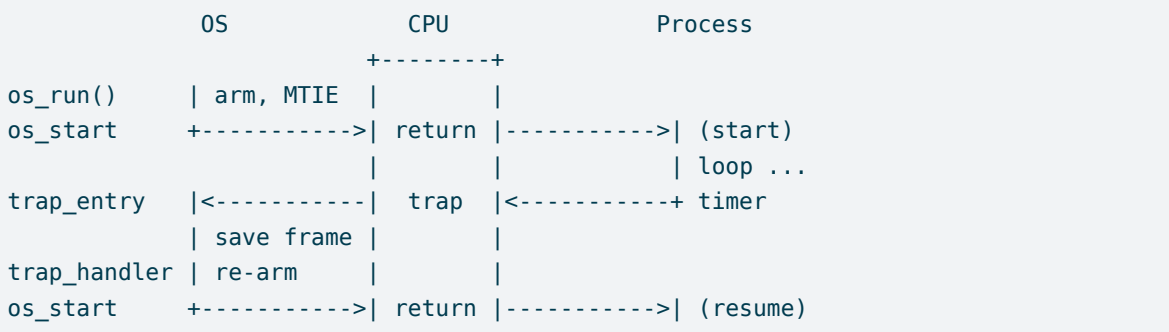
- **Report**, in the Exercise 5 library only: one line with the trap `pc`.
- **Re-arm**: `timer_arm(TIME_SLICE)` moves `mtimecmp` one time slice into the future. This also withdraws the pending request, because `mtime >= mtimecmp` no longer holds.
- **Decide**: `reschedule()` marks the current process `READY` and selects the next `READY` one. With one process, that is the same process, so the same frame is returned.

There is no `pc += 4`. The re-arm is not optional. Without it, `mip.MTIP` would still be set when the handler returns; `mret` sets `mstatus.MIE` to 1, `mie.MTIE` is 1, so all three conditions hold again, and the CPU traps before the spinner executes a single instruction. The OS would run and the process would not.

The parameter of `trap_handler()` is still called `trapped`. For the timer case, read it as "the process that was interrupted".

6.4.5 The lifetime with a timer interrupt

The picture of Lecture 5, "The lifetime in detail", changes in two places only:



`os_run()` arms the timer and sets `mie.MTIE`, and the trap arrow starts at "timer" instead of an `ecall`. The process line has no `ecall` at all: the spinner just keeps computing, and the trap arrow leaves it between two instructions of its loop. From `trap_entry` until `os_start` restores it, the spinner's state is in `p.frame`. The handler bar says "re-arm", not "pc += 4". The process never asked; the OS ran anyway and returned the same frame.

After the spinner exits, the idle process receives the timer interrupts, as Lecture 5 showed for the time after the last exit.

6.5 Preemption

6.5.1 Losing the CPU without yielding

Preemption means that the OS takes the CPU away from a process that did not yield. In our system the only way this happens is the timer interrupt: the process neither causes nor intends the entry into the OS.

	Cooperative (Lecture 5)	Preemptive (Lecture 6)
who causes the entry	the process: `ecall`	the timer
when	where the process chooses	between any two instructions
a `for (;;)` loop	keeps the CPU	loses it every 10 ms

This answers the question of Lecture 4. The answer depends on nothing the process does: the OS arms the timer and turns on the timer switch before the first return, and the next time slice is armed in every timer branch. From now on, no process can keep the OS out for longer than one time slice. That last sentence holds for well-behaved processes only. Everything runs in machine mode, so a process could write `mie` or `mtimecmp` itself; privilege modes, which would prevent this, are outside our model.

Keep this axis apart from the one of Lecture 5. Cooperative versus non-cooperative asks whether the process intends the entry; synchronous versus asynchronous asks whether the current instruction causes it. A timer interrupt is non-cooperative and asynchronous; an illegal instruction is non-cooperative but synchronous.

6.5.2 Interrupt handling versus the scheduling decision

The timer gives the OS control. What the OS does with it is a separate step:

```
interrupt handling     CPU: trap rule          the OS gets control
                       trap_entry: save        the state becomes data
scheduling decision    trap_handler(): which frame to return?
return                 os_start, mret          that frame continues
```

The decision is one line of `trap_handler()`: `return reschedule();`. With one process it has one option, and the same frame continues. Preemption makes a decision possible; it does not make one. Lecture 7 asks whether the handler must return the same frame, and that question only makes sense now that the OS has control without the process's help.

6.5.3 One line per tick

A **tick** is one timer interrupt of the OS. It is not one count of `mtime`, which is why the code uses `delta` and `TIME_SLICE` for `mtime` counts. The Exercise 5 library prints one line per tick taken in a process. Our spinner counts to 1,000,000 and exits with status 7; linked against the Exercise 5 library, it produced, in our build (GCC 13.2, QEMU 8.2):

```
os: starting 1 process
spinner: start
os: tick 1 at 0x800002b8 in process 1
os: tick 2 at 0x800002b8 in process 1
os: tick 3 at 0x800002b8 in process 1
os: tick 4 at 0x800002b6 in process 1
os: tick 5 at 0x800002b6 in process 1
os: tick 6 at 0x800002b6 in process 1
os: tick 7 at 0x800002b4 in process 1
os: tick 8 at 0x800002b4 in process 1
spinner: done, counter = 1000000
os: process 1 exited 7
os: all processes finished
os: 8 timer interrupts
```

The `spinner:` lines come from the process, the `os:` lines from the OS. All eight reported addresses lie in the loop shown above. The last line is the count of timer interrupts, printed when the last process has finished. Addresses and counts are those of our build; yours differ, the structure does not.

6.5.4 Can the process tell it was interrupted?

The same spinner, linked once against the Exercise 4 library and once against the Exercise 5 library:

```
Exercise 4 library              Exercise 5 library
spinner: start                  spinner: start
                                os: tick 1 ... os: tick 8
spinner: done,                  spinner: done,
  counter = 1000000               counter = 1000000
```

The spinner's own lines and its final counter are identical. The tick lines were printed by the OS between two instructions of the spinner; the spinner executed none of its own instructions meanwhile. After each tick, its registers, its memory and its next instruction are as they were before the interrupt.

The process cannot tell that it was interrupted, except that time has passed. It could find out only by measuring time, for example by reading `mtime` before and after its loop and noticing

that more time passed than its own instructions need. This is the only trace preemption leaves, and it is why the trap path must be exact: any register it failed to restore would be a visible trace, and a wrong one.

6.5.5 Reproducible interrupts with `-icount`

Lecture 5 said that an interrupt happens between two instructions, at a place that varies from run to run. Real hardware behaves like that, and so does QEMU by default: its timer follows the time of the host, so the places where the spinner is interrupted change from run to run. That makes asynchronous behaviour hard to observe and impossible to predict exactly.

For the course we start QEMU with an additional flag:

```
qemu-system-riscv64 ... -icount shift=4,sleep=off
```

With `-icount`, QEMU's virtual clock counts executed instructions instead of host time: each instruction advances virtual time by 2^4 ns = 16 ns. `mtime` then depends only on how many instructions ran, and two runs of the same binary are interrupted at the same instructions. A 10 ms time slice is 625,000 instructions; one `mtime` count of 100 ns is about 6 instructions. `sleep=off` is part of the flag on purpose: with plain `shift=4`, the QEMU version of our build environment still interrupted the same binary at different instructions from run to run. Without `-icount`, on a fast host, the spinner may even finish before the first timer interrupt.

`-icount` is our observation tool, not a property of real hardware. Nothing in the OS may depend on where an interrupt lands. The flag makes our observations reproducible; it does not make the system more correct.

6.6 What can go wrong

6.6.1 Ticks without progress

The following output comes from the reference OS with exactly one line of its timer path removed, run with the same spinner. It never reaches `spinner: done`:

```
os: starting 1 process
spinner: start
os: tick 1 at 0x800002b8 in process 1
os: tick 2 at 0x800002b8 in process 1
os: tick 3 at 0x800002b8 in process 1
...                      (tens of thousands of lines)
```

The re-arm is missing from the timer branch. `mtimecmp` stays in the past, so `mip.MTIP` stays set. After every `mret` all three conditions hold at once, and the CPU traps before the spinner executes its next instruction: the same trap `pc` every time, and no progress. The OS is busy, and the process starves.

The conditions table is the tool to find such a defect. Exercise 5 has you diagnose further broken timer paths with your own build.

6.6.2 Our trap model: who does what, with the timer

Lecture 5 summarised the trap model in a table. With the timer it gains one row at the top and changes in two cells:

Step	Who	What
request	CLINT	pending while `mtime >= mtimecmp`
trap	CPU	if enabled: `mepc`, `mcause`; interrupts off; `pc <- mtvec`
save	`trap_entry`	registers and trap `pc` into the frame; OS stack
decide	`trap_handler()`	timer: re-arm, return a frame
restore	`os_start`	registers and `mepc` from the frame
return	CPU, `mret`	`pc <- mepc`; `MIE <- MPIE`, interrupts on

The platform device requests, the CPU takes the interrupt if it is enabled, and the handler re-arms and returns a frame. Everything else is unchanged. For CP2, students write their own `os_run()` and `trap_handler()`: `os_run()` arms the timer and turns on the timer switch, and `trap_handler()` has the timer branch and, after Lecture 7, a scheduler.

The model stays as restricted as in Lecture 5: machine mode only, one trap entry through `mtvec`, no nested traps, since the OS always runs with `mstatus.MIE` = 0, and no privilege separation. Lecture 6 adds exactly one interrupt source, the machine timer of the CLINT. Outside the model: other interrupt sources and the interrupt controllers that collect them, vectored `mtvec`, timers for supervisor mode, and the protection that would keep a process from changing `mie` or `mtimecmp`.

6.7 Key ideas

- The **CLINT** is a platform device: `mtime` counts at 10 MHz, `mtimecmp` holds the time of the next request, and both are memory-mapped registers reached through `volatile` pointers.
- A timer interrupt is **pending** in `mip.MTIP` while `mtime >= mtimecmp`, until the OS writes a larger `mtimecmp`. It is taken only if the timer switch `mie.MTIE` and the global switch `mstatus.MIE` are also on; the OS sets `mie.MTIE` once, and the trap rule and `mret` switch `mstatus.MIE`.

- Because a trap clears `mstatus.MIE`, a timer interrupt never interrupts the OS; one that becomes pending meanwhile is taken right after `mret`.
- A timer interrupt enters through the **same trap path** as an `ecall`. `mcause` has the interrupt bit set and code 7, and the trap `pc` is the instruction not yet executed: it already is the continuation address, so the handler adds nothing.
- The full trap frame of Lecture 5 is what makes this safe: at an interrupt, caller-saved registers hold live values and no calling convention protects them.
- The timer branch of `trap_handler()` re-arms the timer and returns a frame. Without the re-arm, the interrupt stays pending and the process makes no progress.
- **Preemption**: the OS gets the CPU back every time slice without a yield. It makes a scheduling decision possible; with one process, the decision is the same frame.
- `-icount` makes the places of interrupts reproducible for observation; it is not a property of real hardware.

The OS now has control at every tick. Must it give the CPU back to the same process?

Related Exercise

Exercise 5 — The Timer Path runs a never-yielding process against the Exercise 4 and the Exercise 5 library, locates the reported trap `pc` values in the disassembly, and diagnoses three broken libraries from their output.

References

- RISC-V International, *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture*: `mip`, `mie`, `mstatus`, `mcause`, machine timer interrupt.
- RISC-V International, *RISC-V Advanced Core Local Interruptor Specification*: `mtime`, `mtimecmp`.
- QEMU documentation, *TCG Instruction Counting*: `-icount`.
- Arpaci-Dusseau and Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, chapter *Mechanism: Limited Direct Execution*.

A Glossary

Short definitions of terms that the chapters use precisely. Each entry names the lecture where the term first matters. Terms that are easily confused point to each other with *see also*.

Architectural state State that the ISA defines and that every implementation must provide, as opposed to details of the microarchitecture. The `x` registers, the CSRs and the `pc` are all architectural state. The `pc` differs from the others in one respect: it is not addressable. No instruction names it as an operand or stores it to memory; it changes only through control transfers and the trap rule. *See also:* program counter, live `pc`. (Lecture 5)

Automatic storage duration The lifetime of an ordinary local variable, declared inside a function without `static`. The object exists only while that call is active and lives in the call's stack frame; every call, including a recursive one, gets its own copy. When the call returns, or when its stack frame is reused, the object is gone. Example: `struct process p;` inside `main()` in Exercise 2 lies in `main()`'s stack frame at the top of the OS stack. *See also:* static storage duration. (Lecture 4)

Bootstrap function The internal function through which a process is entered for the first time: `process_start()`. It calls the process's entry function and, if that returns, `process_exit(0)`. The initial saved state of a new process points to it, as `ra` in the CP1 context and as `pc` in the trap frame. (Lecture 3)

Caller and callee In a function call, the **caller** is the function that makes the call, the **callee** the function being called. The calling convention splits the work of preserving registers between them. *See also:* calling convention, caller-saved and callee-saved register. (Lecture 3; named explicitly in Lecture 5)

Caller-saved and callee-saved register A **callee-saved** register is one whose value a called function must preserve for its caller: `sp` and `s0`–`s11`. A **caller-saved** register may be destroyed by a call, so the caller keeps any value it still needs elsewhere: `t0`–`t6` and `a0`–`a7`, and `ra`, which every call overwrites. The split is why CP1's `context_switch()` could save only `ra`, `sp` and `s0`–`s11`. *See also:* calling convention. (Lecture 3)

Calling convention The software agreement, defined in the RISC-V ABI, that lets separately compiled functions cooperate about how registers are used: arguments in `a0`–`a7`, the return value in `a0`, the return address in `ra`, and which registers are caller-saved or callee-saved. The compiler guarantees it for the code it generates; hand-written assembly must keep it itself, and the hardware knows nothing of it. A trap is not a call, so the convention gives no guarantee at a trap. (Lecture 3)

CLINT (core-local interruptor) The timer device of QEMU's `virt` machine, a part of the platform, not of the CPU. It holds the memory-mapped registers `mtime` and `mtimecmp` and requests a machine timer interrupt from the CPU while `mtime >= mtimecmp`.

Newer RISC-V documents call it ACLINT. *See also:* `mtime` and `mtimecmp`, platform. (Lecture 6)

Context The saved CPU register state needed to continue the same execution. The CP1 `struct context` holds `ra`, `sp` and `s0`–`s11`, which suffices at a call boundary. From Lecture 5 the trap frame is the context, complete for our model. *See also:* trap frame, context switch. (Lecture 3)

Context switch In CP1: save the current execution's context, restore another saved context, and continue the restored execution. `context_switch()` did both halves in one function called by the running code; from Lecture 5 the trap path saves and the return path restores. (Lecture 3)

Continuation address A saved value saying where an execution continues when it is restored. It is never the live `pc` itself. In CP1 it is `ra` in the context; in the trap frame it is the field `pc`. After an interrupt it equals the trap `pc`; after an `ecall` it is the trap `pc` + 4. *See also:* live `pc`, trap `pc`. (Lecture 4)

Control and status register (CSR) A CPU register outside the general-purpose registers `x0`–`x31`, reachable only through the CSR instructions of the `Zicsr` extension, not through loads and stores. Our trap mechanism uses five: `mtvec` (where traps enter: a base address and a mode field), `mepc` (the trap `pc`), `mcause` (the reason, with the interrupt bit in bit 63), `mstatus` (among others `MIE`, `MPIE`, `MPP`), and `mscratch` (no meaning for the hardware; our OS keeps the current frame's address there). Lecture 6 adds `mie` (one enable bit per interrupt source, among them `MTIE` for the machine timer) and `mip` (the pending bits, among them `MTIP`). The prefix `m` means machine mode. (Lecture 5; Lecture 6)

Cooperative and non-cooperative entry An entry into the OS is **cooperative** if the running execution itself decides to make it, as with `process_yield()` in CP1 or `ecall` in Lecture 5. It is **non-cooperative** if the OS is entered without the execution's intention, as with an illegal instruction or, from Lecture 6, a timer interrupt. This axis is separate from synchronous versus asynchronous: an illegal instruction is synchronous but not cooperative. *See also:* synchronous and asynchronous trap, preemption. (Lecture 3; Lecture 5)

Cross-compilation Building code on one architecture for another. A **cross compiler** runs on the development host, an x86-64 or ARM machine, and produces RV64 code for the target. (Lecture 1)

Development host The machine on which the development tools run: in BVS1 the Ubuntu `bvs1` VM. *See also:* physical host, target. (Lecture 1)

ELF (Executable and Linkable Format) The binary file format we use for both object files and the final executable. An ELF file contains sections, a symbol table and the entry address. (Lecture 1)

Exception A synchronous trap: caused by the instruction being executed, at the same instruction every time the program runs with the same input. The trap `pc` is that instruction. Examples: `ecall` (`mcause` 11 from machine mode), illegal instruction (`mcause` 2). Not an interrupt. *See also:* interrupt, trap. (Lecture 5)

Execution A running sequence of instructions together with the CPU and memory state it needs to continue. Two executions take turns when one stops and another continues; a process is an execution that the OS manages. In CP1 `main()` was an execution too, suspended while the process ran; from Lecture 5 nothing of the OS remains suspended

between traps. *See also:* process. (Lecture 2)

Exit The process finishes, records a result, and returns control to the operating system; it must never continue. From Lecture 5 `process_exit()` makes the system call `exit`. *See also:* yield. (Lecture 2; Lecture 3)

Idle process The process the OS runs when no other process is `READY`. In the reference implementation it is a loop around `wfi` (wait for interrupt); after the last process has exited, the OS returns into its frame. (Lecture 5)

Immediate A value small enough to be encoded inside an instruction, for example in `li` or `addi`. Larger values may need several instructions. (Lecture 4)

Instruction Set Architecture (ISA) The contract between software and a processor: the instructions, registers and behaviour that every implementation provides and that software may rely on. The trap rule and the CSRs belong to it. *See also:* microarchitecture, platform. (Lecture 1)

Interrupt An asynchronous trap: caused by an event independent of the current instruction and taken between two instructions, so the place varies from run to run. The trap `pc` is the instruction that was about to execute and has not executed yet; there is no `+4`. Interrupts are taken only when enabled; on a trap the CPU disables them while the OS saves the process's state. Not an exception: an `ecall` is not an interrupt. *See also:* exception, pending, timer interrupt, trap. (Lecture 4; as a kind of trap in Lecture 5)

Label A name in assembly source, written as `name:`, that names the position of the next instruction or data item. Labels exist only in source code. The assembler turns a named label such as `_start:` or `trap_entry:` into a symbol; a local label such as `1:` in `start.S` is used only within its file and does not appear in the symbol table. `trap_entry:` in `trap.S` names the first instruction of the trap path. Not to be confused with an *output label*, the constant text that a program prints before a value, such as `"process: result: "`. *See also:* symbol. (Lecture 2)

Lifecycle state Says whether an execution may run: `READY`, `RUNNING` or `FINISHED`. The context says how it continues. (Lecture 3)

Linker The tool that combines object files, resolves references between them by symbol name, and places their sections at final addresses in target memory. (Lecture 1; Lecture 2)

Linker script The file that tells the linker where the sections go in target memory and defines symbols such as `_stack_top`. Ours is `linker.ld`; students read it but do not write one. (Lecture 2)

Live `pc` The current execution position of the CPU: the `pc` as architectural state. Software never stores it directly; the CPU copies it into `mepc` on a trap, and `mret` sets it from `mepc`. *See also:* trap `pc`, continuation address. (Lecture 4; named in Lecture 5)

Machine mode The most privileged RISC-V privilege mode (M-mode). Our whole system, OS and processes alike, runs in it, which is why our CSRs carry the prefix `m` and `ecall` produces `mcause` 11. *See also:* privilege mode. (Lecture 5)

Memory-mapped register A device register that has an address in the same address space as RAM, so that an ordinary load or store to that address reads or writes the device instead of memory. In C it is reached through a `volatile` pointer, so that the compiler performs every access as written. Examples: the UART's transmit and status registers at `0x10000000` and `0x10000005`, `mtime` and `mtimecmp`. The platform decides which address leads to RAM and which to a device; on QEMU `virt`, RAM starts at `0x80000000`. CSRs are not memory-mapped: they have no address. *See also:* control

and status register. (Lecture 6)

Microarchitecture How a particular processor implements the ISA internally, for example its pipeline or caches. Software written for the ISA does not depend on it. *See also:* ISA. (Lecture 1)

`mtime` and `mtimecmp` The two 64-bit memory-mapped registers of the CLINT. `mtime` (at `0x0200bff8` on QEMU `virt`) is the current time; the device counts it up from 0 at machine start at 10 MHz, one count every 100 ns, and it never stops. `mtimecmp` (for hart 0 at `0x02004000`) holds a point in time, not a duration: while `mtime >= mtimecmp`, a timer interrupt is pending. `timer_arm(delta)` writes `mtime + delta` into `mtimecmp`. *See also:* CLINT, pending, time slice. (Lecture 6)

Object file A target-specific intermediate file produced by the compiler or assembler, in ELF, that still needs to be linked with others into an executable. (Lecture 1)

Operating system Software that helps turn hardware into a usable computer system by providing abstractions, resource management, coordination, protection, and services for programs and users. Two views: the OS as **resource manager** and the OS as **abstract machine**. (Lecture 1)

OS stack The stack on which the OS's own code runs: the region that the linker script reserves below `_stack_top`. `_start` selects it, and `main()` has run on it since Lecture 2. From Lecture 5 the trap path switches to it explicitly with `la sp, _stack_top`, so every handler run starts at its top; anything that must outlive one handler run must not live there. *See also:* process stack. (Lecture 4)

Pending An interrupt is **pending** when its source has requested it and the request has not been withdrawn; it is **enabled** when the CPU's enable bits allow it, and **taken** when the CPU applies the trap rule. The machine timer interrupt is pending in `mip.MTIP` while `mtime >= mtimecmp`; it stays pending until the OS writes a larger `mtimecmp`, and taking it does not clear it. It is taken only if `mie.MTIE` and `mstatus.MIE` are also set. *See also:* interrupt, `mtime` and `mtimecmp`. (Lecture 6)

Physical host The real computer that provides the CPU, memory and devices, and on which the development VM runs. *See also:* development host, target. (Lecture 1)

Platform The machine around the CPU: memory map and devices such as the UART and, in Lecture 6, the timer. QEMU's `virt` machine is our platform. A platform device raises an interrupt; the CPU's trap rule reacts to it. *See also:* ISA. (Lecture 5)

Polling Asking a device again and again whether it is ready, in a loop, until it is. `uart_putc()` polls the UART's line status register before each character. *See also:* memory-mapped register, UART. (Lecture 6)

Preemption The OS takes control without the running process's cooperation, made possible by a timer interrupt. Interrupt handling and the scheduling decision that may follow are related but distinct steps. In our system from Lecture 6, the timer interrupt enters the OS every time slice; with one process, the scheduling decision returns the same frame. *See also:* cooperative and non-cooperative entry, time slice. (Lecture 4; Lecture 6)

Privilege mode A CPU mode that determines which instructions, CSRs and memory are available. RISC-V defines machine, supervisor and user mode; a real OS runs processes in a less privileged mode, so that they cannot change OS state. Privilege is outside our model: everything runs in machine mode. *See also:* machine mode. (Lecture 5)

Process An execution that the operating system can manage. In our system it combines an

entry function, owned stack storage, a saved context (from Lecture 5 the trap frame), a lifecycle state and an exit status. *See also:* execution. (Lecture 2; Lecture 3)

Process stack The stack of one process: owned stack storage, in our system the array `stack` inside `struct process`, which `sp` selects while the process runs. "Process stack" names the role of the memory, not an ELF section: the array lies wherever the process object lies. *See also:* OS stack. (Lecture 3; Lecture 4)

Program counter (`pc`) Identifies the instruction to execute. It is CPU state, not memory, and not one of the general-purpose registers `x0`–`x31`. *See also:* architectural state, live `pc`. (Lecture 4)

Pseudoinstruction Assembler notation that is not itself an instruction but is translated into one or more real instructions. Examples: `li`, `mv`, `j`, `ret` (`jalr x0, 0(ra)`), and the CSR forms `csrr`, `csrw` and `csrs`, which become `csrrs` or `csrrw`. (Lecture 5)

QEMU An emulator and virtualizer. It runs as an ordinary program in the development VM and emulates the RV64 target machine; we start it without firmware (`-bios none`). (Lecture 1)

Resume Continue a paused execution where it stopped, as opposed to starting it again from its entry function. From the CPU's point of view, resuming and starting a process are the same operation: a return into a saved frame. (Lecture 3)

Return path The second half of the provided `trap.S`, from the label `os_start` to `mret`: it makes `mscratch` point to the frame it restores, writes the frame's `pc` into `mepc`, loads the registers and executes `mret`. It has two ways in: after every trap, by falling through from `call trap_handler`, and once from `os_run()` for the first start. *See also:* trap path. (Lecture 5)

Return-address register `ra` (`x1`): a `call` stores the location to which execution returns, and `ret` jumps there. It is the continuation of a call. (Lecture 3)

RV64 The 64-bit form of RISC-V, the ISA of our target. We build for `rv64imac_zicsr`: the base integer instructions, multiplication and division, atomics and compressed instructions, plus the CSR instructions. (Lecture 1)

Section A named part of an object file or executable that groups content of one kind; the linker places sections in memory according to the linker script. `.text` holds instructions, `.rodata` constant data, `.data` initialised data, and `.bss` zero-initialised data, which takes no space in the file. (Lecture 2)

Stack Runtime memory for the data of active function calls: saved call information, saved registers, local variables. Function instructions remain in `.text`. A stack grows toward lower addresses. *See also:* stack frame, OS stack, process stack. (Lecture 2; Lecture 3)

Stack frame The call data of one active call on a stack, for example its saved `ra`, saved registers and local variables. Not a trap frame. *See also:* trap frame. (Lecture 4)

Stack pointer (`sp`) Register `x2`; selects the current position in the stack in use. Which stack is in use is decided by the value in `sp`: a trap does not change it. (Lecture 2; Lecture 4)

Static storage duration The lifetime of global variables and of variables declared `static`, also inside a function. The object exists exactly once, for the whole run, independent of any stack frame and of which functions are active; it is initialised once, before the program runs. Its section depends on the initial value: `.bss` for no initialiser or zero, `.data` for a non-zero initialiser, `.rodata` for `const` data. `static` inside a function changes only where and how long the object lives; the name stays visible only

inside that function. Example: `static struct process p;` in `main()` lies in `.bss` and survives every trap. *See also:* automatic storage duration. (Lecture 4; `static p` in Lecture 5)

Symbol An entry in the symbol table of an object file or an ELF file: a name with a value, usually an address, plus a type and a visibility. `nm -n` lists the symbols of a program sorted by address. Symbols come from labels in assembly, from C functions and from C variables with static storage, for which the compiler emits labels, or from assignments in the linker script. Example: `trap_entry` comes from a label, `main` from a C function, `_stack_top` from `linker.ld`. The linker resolves references between separately translated parts by symbol name, and fixes the final address of every symbol when it places the sections. *See also:* label. (Lecture 2; `nm -n` in Lecture 4)

Synchronous and asynchronous trap A **synchronous** trap is caused by the instruction being executed and is reproducible: an exception. An **asynchronous** trap is caused by an event outside the instruction stream and is taken between two instructions, wherever the program happens to be: an interrupt. *See also:* exception, interrupt, cooperative and non-cooperative entry. (Lecture 5)

System call A request to the OS made with `ecall`. In our convention `a7` selects the service, `a0` and `a1` carry arguments, and the OS may return a result in `a0`: `exit` is 0, `yield` is 1. The system call is the request, not the C function that makes it: `process_yield()` makes the system call `yield`. (Lecture 5)

Target The machine for which we build the program: an RV64 machine, emulated by QEMU. *See also:* development host, physical host. (Lecture 1)

Tick One timer interrupt of the OS, as in the Exercise 5 library's lines `os: tick N at <pc> in process 1`. Not one count of `mtime`: that is why `timer_arm()` takes a `delta` and the time slice is called `TIME_SLICE`, both in `mtime` counts. *See also:* time slice, timer interrupt. (Lecture 6)

Time slice How long the OS lets a process run until the next timer interrupt; other books call it a *quantum*. In our system 10 ms, `TIME_SLICE` = `TIMER_HZ / 100` = 100,000 `mtime` counts. *See also:* tick, preemption. (Lecture 6)

Timer interrupt An interrupt raised by the platform's timer when a programmed time is reached (`mcause` 7 with the interrupt bit set). It lets the OS regain control without a voluntary yield. In our system the CLINT requests it when `mtime` reaches `mtimecmp`; the trap `pc` is the instruction not yet executed, and the handler re-arms the timer before it returns a frame. *See also:* pending, tick. (Lecture 4; implemented in Lecture 6)

Toolchain The set of cooperating build tools: compiler, assembler and linker, here for the RV64 target. (Lecture 1)

Trap A transfer of control to an OS-defined handler that the CPU performs itself. On a trap the CPU records where (`mepc`) and why (`mcause`), turns interrupts off (`MPIE <- MIE`, `MIE <- 0`) and jumps to `mtvec`. The umbrella term for exceptions and interrupts. *See also:* exception, interrupt, return path. (Lecture 5)

Trap frame The complete saved integer context of an execution that trapped, as our teaching model requires it: `struct trap_frame`, with the 31 writable integer registers in `regs[1]`–`regs[31]` and the continuation address in `pc`. It is the field `frame` of `struct process`, not on any stack; the name comes from systems that push it onto a stack. *See also:* stack frame, context. (Lecture 5)

Trap handler The OS code that runs after a trap. In the course it is the C function `trap_handler()`, which the trap path calls with the frame of the process that trapped and which returns the frame to restore: the same one to resume, another one to schedule. In lifetime diagrams, "handler" labels the whole OS run between a trap and a return. *See also:* trap path, return path. (Lecture 5)

Trap path The first half of the provided `trap.S`, from the label `trap_entry` to `call trap_handler`: it saves all registers and the trap `pc` into the frame that `mscratch` points to, switches to the OS stack and calls `trap_handler()`. The CPU jumps to it via `mtvec`; nobody calls it. *See also:* return path. (Lecture 5)

Trap `pc` Course term, not a RISC-V name: the `pc` value that the CPU records in `mepc` on trap entry. For an exception it is the causing instruction, for an `ecall` the `ecall` itself; for an interrupt it is the next instruction, not yet executed. *See also:* live `pc`, continuation address. (Lecture 5)

UART The serial device of our platform through which all output of the course is printed; on QEMU `virt` its registers start at `0x10000000`. `uart_putc()` polls its line status register and stores each character into its transmit register, with ordinary loads and stores. *See also:* memory-mapped register, polling, platform. (Lecture 6)

Yield The running process voluntarily returns control to the operating system while remaining able to resume later. From Lecture 5 `process_yield()` makes the system call `yield`. *See also:* exit. (Lecture 2; Lecture 3)